

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

OmniAI Gateway

Relatório Final de Projeto

Guilherme Coutinho

Nº 50467

A50467@alunos.isel.pt

André Nunes

Nº 51766

A51766@alunos.isel.pt

Orientador: Pedro Félix

pedro.felix@isel.pt

Engenharia Informática e de Computadores

29 de maio de 2026

Resumo

A ausência de um protocolo comum na comunicação com Modelos de Linguagem de Grande Escala (LLMs) obriga as aplicações a integrarem diferentes interfaces de acesso e implementações particulares para a invocação de ferramentas externas. Atualmente cada fornecedor (como OpenAI [1], Anthropic [2] e Gemini [3]) impõe as suas próprias estruturas de dados, ciclos de vida de *streaming* e padrões para a invocação de ferramentas (*tool calling*) [4] [5]. Para resolver esta fragmentação tecnológica, este projeto propõe o desenvolvimento do **OmniAI SDK**, um *Software Development Kit* construído em Kotlin Multiplatform (KMP) [6]. O objetivo principal desta biblioteca é fornecer a infraestrutura base necessária para instanciar e configurar de forma rápida uma **OmniAI Gateway** (um ponto de entrada centralizado dedicado ao acesso de inferência, segurança, observabilidade e outros). O **OmniAI SDK** é baseado nos princípios da Arquitetura Hexagonal (*Ports and Adapters*) [7] [?], este isola a lógica de domínio e suporta a comunicação bidirecional compatível com os padrões das APIs de referência, nomeadamente OpenAI [1], Anthropic [2] e Gemini [3]. O sistema implementa uma *pipeline* extensível e configurável, inspirada no modelo de filtros e cadeia de filtros dos Servlets [8] [9], que executa *interceptors* pré-construídos para autorização, autenticação, métricas, *rate limiting*, *circuit breaker* e *fallback*, podendo ser adicionados mais *interceptors* customizados pelo utilizador do **OmniAI SDK**. Adicionalmente, o *Software Development Kit* prepara um módulo *MCP Broker* para evoluir a **OmniAI Gateway** de uma *gateway* de inferência e torná-la numa *AI Gateway* completa, adicionando a gestão de ferramentas externas à *OmniAI Gateway* utilizando o *Model Context Protocol(MCP)* [10]. Distribuído de forma multiplataforma para ambientes JVM, via Maven, e Node.js/TypeScript, via NPM, o **OmniAI SDK** foi validado através da construção de uma **OmniAI Gateway**, um *Proof of Concept* que utiliza a biblioteca para demonstrar a criação de uma *gateway* real. A avaliação demonstra que a solução minimiza o acoplamento tecnológico e o esforço de integração, garantindo resiliência, escalabilidade e centralização na utilização de clientes de IA, como o Claude Code [11], entre outros.

Abstract

The absence of a common protocol for communicating with Large Language Models (LLMs) forces applications to integrate diverse access interfaces and bespoke implementations for external tool invocation. Currently, each provider (such as OpenAI [1], Anthropic [2], and Gemini [3]) imposes its own data structures, streaming lifecycles, and tool calling patterns [4] [5]. To address this technological fragmentation, this project proposes the development of the **OmniAI SDK**, a Software Development Kit built using Kotlin Multiplatform (KMP) [6]. The primary objective of this library is to provide the foundational infrastructure required to rapidly instantiate and configure an **OmniAI Gateway** (a centralized entry point dedicated to inference access, security, observability, and more). Based on the principles of Hexagonal Architecture (*Ports and Adapters*) [7] [?], the **OmniAI SDK** isolates domain logic and supports bidirectional communication compatible with reference API standards, namely OpenAI [1], Anthropic [2], and Gemini [3]. The system implements an extensible and configurable *pipeline*, inspired by the Servlet filter and filter chain model [8] [9], which executes pre-built *interceptors* for authorization, authentication, metrics, *rate limiting*, *circuit breaker*, and *fallback*, while allowing the user of the **OmniAI SDK** to add custom interceptors. Additionally, the Software Development Kit features an MCP Broker module to evolve the **OmniAI Gateway** from a simple inference gateway into a comprehensive *AI Gateway*, adding external tool management to the **OmniAI Gateway** using the *Model Context Protocol (MCP)* [10]. Distributed as a multiplatform solution for JVM environments, via Maven, and Node.js/TypeScript, via NPM, the **OmniAI SDK** was validated through the construction of an **OmniAI Gateway**, a Proof of Concept that leverages the library to demonstrate a real-world gateway deployment. The evaluation demonstrates that the solution minimizes technological coupling and integration effort, ensuring resilience, scalability, and centralization across AI clients, such as Claude Code [11], among others.

Índice

1	Introdução	4
1.1	Contexto	4
1.2	Motivação e Definição do Problema	5
1.3	Objetivos e Proposta de Solução	6
1.4	Estrutura do Documento	7
2	Estudo Prévio	8
2.1	Análise das APIs de Inferência	8
2.1.1	Construção do Pedido	9
2.1.2	Processamento da Resposta	11
2.1.3	Divergências na Padronização de Esquemas de Validação	13
2.1.4	Conclusão do Estudo das APIs de Inferência	13
2.2	Model Context Protocol (MCP)	15
3	Modelação do Domínio Comum	17
3.1	Visão Global do Domínio	17
3.2	Estrutura do Domínio Comum	18
3.2.1	Request	18
3.2.2	Response	20
3.3	TypedMap e Extensibilidade do Domínio	23
4	Arquitetura e Desenho do Sistema	25
4.1	Arquitetura Hexagonal (Ports & Adapters)	27
4.1.1	Contracts	28
4.1.2	Inbounds	28
4.1.3	Outbounds	28
4.1.4	HttpTransportClient	29
4.1.5	Dispatcher	29
4.2	Ecossistema do SDK	30
5	Implementação e Evolução Técnica	32
5.1	Stack Tecnológica e Metodologias	32
5.2	A Primeira Iteração da Gateway	32
5.3	Decisões de Desenho e Alternativas Descartadas	33

5.4	Evolução Arquitetural: De <i>Inference Service</i> a <i>Dispatcher</i>	34
5.5	Evolução para a Pipeline de Interceptors	34
5.6	Arquitetura Interna dos Interceptors	36
5.6.1	Interceptor de Telemetria e Observabilidade	37
5.6.2	Interceptor de Autenticação	39
5.6.3	Interceptor de Autorização	41
5.6.4	Interceptor de Rate Limiting	42
5.6.5	Interceptor de Circuit Breaker	43
5.6.6	Interceptor de Fallback	43
5.6.7	Interceptor de Logging e Tracing	44
5.7	MCP Broker e Ferramentas Externas	44
5.8	Distribuição Multiplataforma	45
6	Provas de Conceito (PoCs) e Validação	47
6.1	Validação do POC OmniAI Gateway	47
6.2	Validação com REST Client	48
6.3	Validação de Segurança com Logto	48
6.4	Validação de Observabilidade	48
6.5	Validação com Aplicações Cliente	49
6.6	Limites da Validação	49
7	Conclusões e Trabalho Futuro	50
7.1	Conclusões	50
7.2	Melhorias Futuras e Escalabilidade	50
A	Exemplos JSON das APIs de Inferência	51
A.1	Pedidos (<i>Requests</i>)	51
A.1.1	OpenAI (formato compatível)	51
A.1.2	Anthropic	53
A.1.3	Google Gemini	55
A.2	Respostas (<i>Responses</i>)	56
A.2.1	OpenAI (formato compatível)	56
A.2.2	Anthropic	58
A.2.3	Google Gemini	60
B	Histórico de Planeamento e Riscos	63
B.1	Riscos	63
B.2	Planeamento	65

Lista de Figuras

1.1	Acoplamento direto entre cliente e um fornecedor de inferência	5
1.2	Multiplicação de conexões com chaves de API descentralizadas	5
1.3	Cliente decide trocar de fornecedor	5
1.4	Centralização com OmniAI Gateway	6
2.1	Estruturas JSON esquemáticas e simplificadas de pedidos para cada fornecedor. .	10
2.2	Estruturas JSON esquemáticas e simplificadas de respostas para cada fornecedor.	12
3.1	Fluxo de normalização entre contratos externos e domínio comum (OmniAi-SDK)	17
3.2	Estrutura de classes do <code>CommonRequest</code>	19
3.3	Estrutura de classes do <code>CommonResponse</code>	21
3.4	Utilização da <code>TypedMap</code> para transportar atributos extensíveis	24
4.1	Arquitetura <i>top-level</i> : O OmniAI SDK utilizado para instanciar uma OmniAI Gateway (configurada como PoC).	26
4.2	Visão da integração entre Ktor, inbounds, pipeline, interceptors, outbounds e HTTP client no OmniAI SDK	26
5.1	Visão conceitual da pipeline de interceptors	35
5.2	Fluxo interno de processamento e lógica de observabilidade para pedidos unários e streaming	38
5.3	Observabilidade e métricas operacionais da OmniAI Gateway	39
5.4	Sequência OIDC: descoberta, JWKS e cache de chaves	40
5.5	Fluxo interno do interceptor de autenticação	41
5.6	Arquitetura do interceptor de autorização: separação entre PEP e PDP	42
5.7	Representação do ciclo de <i>tool calling</i> e interação com ferramentas externas . . .	45
6.1	Cenários de validação do POC OmniAI Gateway	47

Capítulo 1

Introdução

1.1 Contexto

Os Modelos de Linguagem de Grande Escala (*Large Language Models – LLMs*) [12] tornaram-se componentes relevantes no desenvolvimento de aplicações baseadas em inteligência artificial. Um LLM é um modelo treinado com grandes volumes de texto, capaz de interpretar linguagem natural, gerar respostas e apoiar tarefas como sumarização, classificação, programação assistida, pesquisa semântica. Uma funcionalidade também oferecida por estes modelos é o *tool calling* [4] [5]. O *tool calling* permite que os modelos não se limitem ao seu conhecimento estático, permitindo-lhes realizar um pedido de interação com sistemas externos através de uma resposta estruturada. Este pedido obedece a um formato específico, definido previamente no momento em que as ferramentas são enviadas a API de inferência.

O ciclo de vida de um modelo divide-se em duas fases principais: treino e inferência. O treino corresponde ao processo de aprendizagem a partir de grandes conjuntos de dados, exigindo elevados recursos computacionais. A inferência corresponde à utilização do modelo já treinado para gerar respostas a novos pedidos. Atualmente, a maioria das aplicações integra LLMs através de APIs de inferência disponibilizadas por fornecedores externos.

A comunicação com estas APIs é normalmente realizada usando o protocolo HTTP com representação em JSON, tanto nos pedidos como nas respostas. Antes de serem processados pelos modelos, os textos são convertidos em *tokens*, pequenas unidades de informação que permitem ao modelo representar e manipular sequências de entrada e saída [13]. Como a resposta é gerada de forma incremental, muitas APIs suportam *streaming*, permitindo enviar fragmentos da resposta ao cliente à medida que são produzidos.

Existem vários fornecedores relevantes, incluindo OpenAI [1], Anthropic [2], Google Gemini [3] e Groq [14]. Embora todos disponibilizem APIs HTTP e estruturas JSON, cada fornecedor define interfaces próprias, campos específicos, regras de autenticação e modelos distintos para *tool calling*. A integração direta com vários fornecedores obriga, por isso, as aplicações a conhecerem detalhes internos de cada API.

Ao mesmo tempo, o uso de LLMs evoluiu de interações simples para sistemas baseados em agentes (*agentic workflows*) [15]. Nestes sistemas, o modelo não se limita a devolver texto, pode também devolver instruções para chamadas a ferramentas externas, interpretar resultados intermédios e produzir uma resposta final com base nesses dados. O *Model Context Protocol*

(MCP) surgiu como proposta de normalização para a comunicação entre modelos, aplicações e ferramentas externas [10].

1.2 Motivação e Definição do Problema

As Figuras 1.1, 1.2 e 1.3 ilustram o acoplamento direto entre clientes e fornecedores de inferência, bem como a dificuldade que os clientes tem em trocar de fornecedor devido a todo acoplamento que existe.

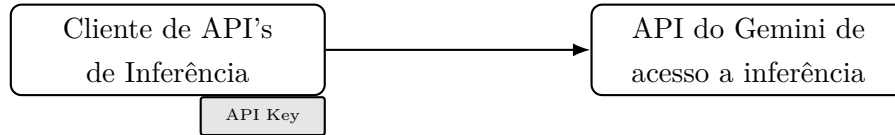


Figura 1.1: Acoplamento direto entre cliente e um fornecedor de inferência

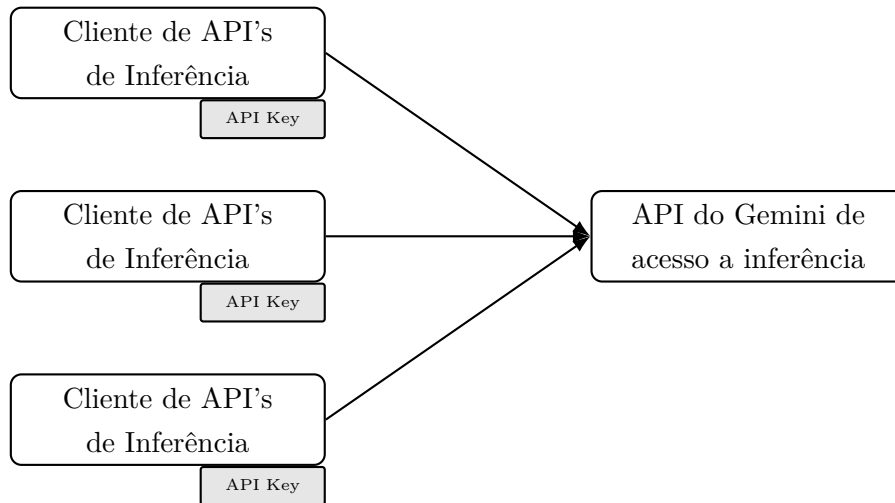


Figura 1.2: Multiplicação de conexões com chaves de API descentralizadas

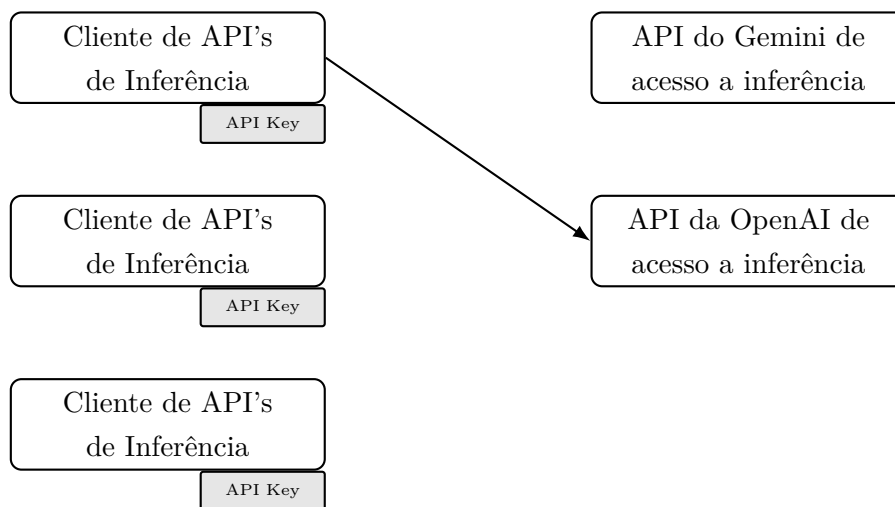


Figura 1.3: Cliente decide trocar de fornecedor

A existência de múltiplos fornecedores de LLMs e a ausência de um protocolo comum implicam diferentes interfaces de acesso aos modelos, estruturas específicas de pedidos e respostas, modelos distintos de contabilização de tokens, custos variáveis e implementações particulares para invocação de funções externas (ver Figuras 1.1–1.3). Por exemplo, a API da OpenAI [1] organiza a interação em torno de chat completions e tool calls; a API da Anthropic [2] utiliza uma arquitetura baseada em messages e blocos de conteúdo; e a API do Gemini [3] estrutura a geração através de contents, parts, function declarations e configurações de geração próprias.

Estas diferenças criam acoplamento entre as aplicações cliente e os fornecedores, denominado de *vendor lock-in*. Sempre que uma aplicação pretende trocar de modelo, suportar mais do que um fornecedor, adicionar um mecanismo de *fallback* ou integrar ferramentas externas, torna-se necessário duplicar ou reimplementar lógica de tradução, autenticação, tratamento de erros e interpretação de respostas. Esta duplicação ou reimplementação aumenta a complexidade e dificulta a evolução do sistema, um problema recorrente em sistemas que combinam lógica de aplicação com integrações externas especializadas [16].

Para além da fragmentação dos contratos, existem preocupações operacionais e de segurança. Uma aplicação que comunica diretamente com vários fornecedores precisa de gerir chaves de API, quotas, limites de utilização, métricas, auditoria, políticas de acesso e proteção contra falhas de terceiros. Sem uma camada intermédia, estas preocupações ficam espalhadas por vários pontos do sistema, tornando mais difícil aplicar regras consistentes de autenticação, autorização, observabilidade e resiliência.

O problema central deste projeto é, assim, a ausência de uma camada de abstração capaz de separar as aplicações cliente das particularidades de cada fornecedor de LLMs, mantendo simultaneamente suporte para segurança, monitorização, *tool calling* e integração dinâmica com ferramentas externas.

1.3 Objetivos e Proposta de Solução

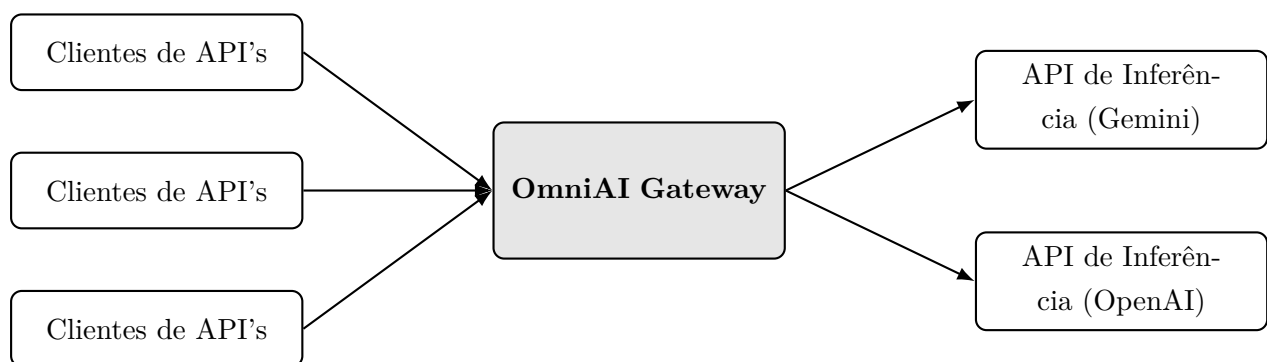


Figura 1.4: Centralização com OmniAI Gateway

A solução proposta evoluiu para a construção de um **OmniAI SDK**, isto é, uma biblioteca reutilizável que permite instanciar uma **OmniAI Gateway**. A Gateway funciona como ponto de entrada único para aplicações cliente, centralizando o acesso a APIs de inferência, ferramentas

externas e mecanismos transversais de segurança e operação. Esta abordagem aproxima-se do padrão de API Gateway [17] e, no contexto específico de modelos de IA, do padrão de AI Gateway enquanto camada centralizada de acesso a modelos [18,19]. Embora existam soluções relacionadas, como LiteLLM e OpenRouter [20,21], o foco deste projeto está na construção de um SDK extensível, configurável e validado por uma Gateway Proof of Concept própria.

O **OmniAI SDK** foi desenvolvido em Kotlin Multiplatform, permitindo partilhar o núcleo de domínio entre targets diferentes e preparar a distribuição para JVM e JavaScript/Node.js [22]. Esta decisão torna possível disponibilizar a biblioteca em ecossistemas distintos: via Maven para aplicações JVM/Kotlin e via NPM para ambientes Node.js/TypeScript.

A arquitetura do **OmniAI SDK** foi organizada segundo os princípios da Arquitetura Hexagonal, também conhecida como *Ports and Adapters* [7]. O domínio comum fica isolado de contratos externos, enquanto os *inbound adapters* traduzem pedidos de clientes compatíveis com OpenAI, Anthropic ou Gemini para o modelo comum, e os *outbound adapters* traduzem o modelo comum para chamadas reais aos fornecedores.

Para tornar a solução configurável e escalável, foi criada uma *pipeline* interna de *interceptors*, inspirada no modelo de filtros e cadeia de filtros dos Servlets [8] [9]. Esta *pipeline* permite injetar comportamentos específicos tanto antes como após a chamada ao fornecedor de inferência ou ao próximo *interceptor* na *pipeline*, incluindo autenticação, autorização, métricas, *rate limiting*, *circuit breaker*, *fallback* e *logging*.

Para garantir que a **OmniAI Gateway** atue como uma peça de infraestrutura completa de inteligência artificial, a arquitetura incorpora um *MCP Broker*. Este módulo permite a exposição e centralização de múltiplas ferramentas externas através do *Model Context Protocol* (MCP) [10].

1.4 Estrutura do Documento

Este documento encontra-se organizado de forma progressiva, partindo do problema de integração e avançando até à validação da solução construída. O Capítulo 2 apresenta o estudo prévio: primeiro a análise comparativa das APIs de inferência e depois o estudo do *Model Context Protocol*. O Capítulo 3 descreve a modelação do domínio comum, explicando como os conceitos observados no estudo foram transformados em estruturas como **CommonRequest**, **CommonResponse**, **CommonResponseEvent**, **CommonTool** e **TypedMap**. O Capítulo 4 descreve a arquitetura do **OmniAI SDK** e do **Proof of concept**, incluindo a visão top-level, a Arquitetura Hexagonal, os módulos, os *contracts*, os *inbounds*, os *outbounds*, o *dispatcher*, a DSL e o cliente HTTP. O Capítulo 5 detalha a implementação técnica e a evolução do sistema, com foco na separação entre **OmniAI SDK** e **OmniAI Gateway**, na *pipeline* de *interceptors*, nos mecanismos de segurança, observabilidade e resiliência, no MCP Broker e na distribuição multiplataforma. O Capítulo 6 apresenta as provas de conceito efetivamente realizadas no Proof of concept OmniAI Gateway. O Capítulo 7 sintetiza as conclusões, limitações e linhas de evolução futura. O Capítulo 8 regista o histórico de planeamento e os principais riscos considerados durante o desenvolvimento.

Capítulo 2

Estudo Prévio

2.1 Análise das APIs de Inferência

O estudo inicial comparou APIs de inferência, nomeadamente os líderes de mercados, como as da OpenAI [1], Anthropic [2] e Gemini [3], para este estudo, utilizou-se o Ollama para analisar a API da Anthropic, o Groq para a da OpenAI, e o AI Studio da própria Google para avaliar a API do Gemini.

Este estudo não teve apenas um papel de descrever as APIs, mas sim servir como base e ponto de partida para observar os conceitos que podiam ser normalizados no OmniAI SDK e os pontos onde a normalização teria de preservar extensões específicas de cada fornecedor. Para esse efeito, foram executados e documentados pedidos de geração simples, pedidos com *streaming*, pedidos com *tool calling*, respostas com contabilização de *tokens* e casos de configuração de geração. É de notar que o uso de vídeos, imagens e outros formatos suportados por estas APIs seria interessante, mas aumentaria demasiado o escopo do estudo.

O estudo das APIs de inferência mostrou que estas partilham de diversos conceitos semelhantes, mas permitem a utilização dos mesmos de maneira diferente. A OpenAI organiza a interação em torno de `chat/completions`, com `messages`, `choices`, `tool_calls` e eventos incrementais baseados em `delta` [1]. A Anthropic usa uma API de `messages`, onde o `system prompt` está no nível de topo, o campo `max_tokens` é obrigatório, e o conteúdo da resposta é uma lista de blocos como `text`, `tool_use` ou `thinking` [2]. A Gemini modela o pedido como `contents` compostos por `parts`, agrupa parâmetros em `generationConfig`, representa ferramentas por `functionDeclarations` e introduz conceitos próprios como *thought signatures* e `thinkingConfig` [3]. Ao longo deste capítulo as diferenças e semelhanças iram ficar mais claras.

2.1.1 Construção do Pedido

Construção do Pedido	OpenAI	Anthropic	Gemini
Autenticação	Cabeçalho Authorization: Bearer <TOKEN> .	Cabeçalho x-api-key e obrigatoriedade de anthropic-version .	Cabeçalho x-goog-api-key ou parâmetro ?key= no URI.
URI (<i>Endpoint</i>)	Fixo por tarefa (e.g., .../v1/chat/completions).	Fixo para mensagens (.../v1/messages).	Dinâmico, acoplando modelo e método (.../models/{model}:{method}).
Identificação do Modelo	Definido via propriedade "model" na raiz do <i>payload</i> JSON.	Definido via propriedade "model" na raiz do <i>payload</i> JSON.	Injetado dinamicamente no próprio URI de chamada HTTP.
Estrutura do Histórico	<i>Array messages</i> contendo objetos com role e content .	<i>Array messages</i> contendo objetos com role e content .	<i>Array contents</i> contendo objetos com role e <i>array parts</i> .
Conteúdo da Mensagem	O content pode ser <i>string</i> simples ou <i>array</i> polimórfico.	O content exige estritamente um <i>array</i> de blocos estruturados.	O conteúdo divide-se num <i>array parts</i> de objetos tipados.
Instruções de Sistema	Tratadas como mensagem inicial com role: "system" .	Declaradas na propriedade isolada "system" na raiz do <i>payload</i> .	Declaradas no objeto isolado "system_instruction" na raiz.
Parâmetros de Geração	Propriedades soltas na raiz (e.g., temperature , max_tokens).	Propriedades soltas na raiz ("max_tokens" é obrigatório).	Agrupados num único objeto configuracional ("generationConfig").
Declaração de Ferramentas	Propriedade tools[].function contendo parameters .	Propriedade tools[] contendo input_schema .	Propriedade tools[].functionDeclarations contendo parameters .
Ativação de <i>Streaming</i>	Propriedade "stream": true na raiz do corpo JSON.	Propriedade "stream": true na raiz do corpo JSON.	URI distinto (streamGenerateContent) com parâmetro ?alt=sse ; não existe campo no corpo.
Configuração de Raciocínio	Suporte nativo e automático na família de modelos o1/o3 ; sem campo dedicado de ativação.	Objeto "thinking" na raiz, com type e budget_tokens para controlar o orçamento.	Objeto thinkingConfig dentro de generationConfig , com opções como thinkingLevel e include_thoughts .
Saídas Estruturadas	Controladas via "response_format" na raiz (e.g., {"type": "json_object"}).	Implementadas forçando uma ferramenta dedicada via tool_choice para extrair dados.	Definidas via responseMimeType e responseJsonSchema dentro de generationConfig .

Tabela 2.1: Mapeamento técnico da estrutura de pedidos e configuração (*Requests*).

Com base nos pedidos executados durante o estudo, foi possível identificar os campos JSON mais relevantes de cada fornecedor. A Figura 2.1 apresenta os esqueletos estruturais dessas mensagens, evidenciando as diferenças de organização e de nomenclatura entre os três fornecedores. Os exemplos JSON completos dos pedidos encontram-se no Apêndice A.1.

OpenAI Request	Anthropic Request	Gemini Request
<pre>{ "model": "gpt-4o", "messages": [{ "role": "system", "content": "..." }, { "role": "user", "content": "..." }], "temperature": 0.7, "max_tokens": 150, "tools": [{ "type": "function", "function": { "name": "func", "parameters": {...} } }], "tool_choice": "auto", "stream": false }</pre>	<pre>{ "model": "claude-3-5-sonnet", "system": "...", "messages": [{ "role": "user", "content": [{ "type": "text", "text": "..." }] }], "max_tokens": 150, "temperature": 0.7, "tools": [{ "name": "func", "input_schema": {...} }], "tool_choice": { "type": "auto" } }</pre>	<pre>URI: models/{model}:{method} { "contents": [{ "role": "user", "parts": [{"text": "..."}] }], "systemInstruction": { "parts": [{"text": "..."}] }, "generationConfig": { "temperature": 0.7, "maxOutputTokens": 150 }, "tools": [{ "functionDeclarations": [{ "name": "func", "parameters": {...} }] }] }</pre>

Figura 2.1: Estruturas JSON esquemáticas e simplificadas de pedidos para cada fornecedor.

A análise dos campos de pedido revela que, embora os três fornecedores partilhem conceitos equivalentes, cada um organiza o corpo JSON de forma distinta. A OpenAI [1] e a Anthropic [2] colocam a maioria dos parâmetros de configuração como propriedades soltas na raiz do objeto, enquanto a Gemini agrupa-os dentro de `generationConfig` [3]. A diferença mais notável está na identificação do modelo. Neste caso, a OpenAI e a Anthropic incluem-na no corpo do pedido, enquanto a Gemini a incorpora diretamente no URI da chamada HTTP, tornando o *endpoint* dinâmico. O *system prompt* também diverge consideravelmente: a OpenAI trata-o como uma mensagem com o papel `system` [1], a Anthropic eleva-o a uma propriedade de topo [2], e a Gemini define-o através de um objeto `system_instruction` separado [3]. Estas diferenças tornam evidente que uma tradução superficial dos nomes dos campos seria insuficiente para construir uma abstração e um domínio comum que seja capaz de cobrir os três fornecedores de inferência.

2.1.2 Processamento da Resposta

Processamento da Resposta	OpenAI	Anthropic	Gemini
Estrutura Raiz da Resposta	Objeto <code>choices[]</code> contendo <code>message</code> com <code>role</code> e <code>content</code> .	Diretamente na raiz: <code>role</code> , <code>content[]</code> , <code>stop_reason</code> e <code>usage</code> .	Objeto <code>candidates[]</code> contendo <code>content</code> com <code>role</code> e <code>parts[]</code> .
Invocação pelo Modelo (<i>Tool Call</i>)	Retorna <code>tool_calls[]</code> com <code>arguments</code> em <i>string</i> JSON serializada.	Retorna um bloco <code>type: "tool_use"</code> com <code>input</code> em JSON nativo.	Retorna uma <code>part</code> contendo <code>functionCall</code> com <code>args</code> (JSON nativo).
Resposta da Ferramenta	Adicionada como nova mensagem com <code>role: "tool"</code> e <code>tool_call_id</code> .	Adicionada sob <code>role: "user"</code> num bloco <code>type: "tool_result"</code> com <code>tool_use_id</code> .	Adicionada como nova mensagem contendo <code>functionResponse</code> com <code>name</code> e <code>response</code> .
Razão de Paragem	Campo <code>finish_reason</code> com valores como <code>"stop"</code> , <code>"tool_calls"</code> ou <code>"length"</code> .	Campo <code>stop_reason</code> com valores como <code>"end_turn"</code> , <code>"tool_use"</code> ou <code>"max_tokens"</code> .	Campo <code>finishReason</code> com valores como <code>"STOP"</code> ou <code>"MAX_TOKENS"</code> (notação em maiúsculas).
Saídas Estruturadas	Controladas via propriedade <code>"response_format"</code> na raiz.	Implementadas forçando uma ferramenta dedicada via <code>tool_choice</code> .	Definidas via <code>responseMimeType</code> e <code>responseJsonSchema</code> .
Comportamento <i>Streaming</i>	Ativado via <code>stream: true</code> ; eventos SSE com formato <code>delta</code> .	Ativado via <code>stream: true</code> ; eventos SSE fortemente tipados (<code>message_start</code> , <code>content_block_delta</code> , etc.).	Consumido através de um <code>endpoint</code> distinto com parâmetro <code>?alt=sse</code> ; cada evento contém um objeto de resposta parcial completo.
Metadados de Raciocínio	Suporte nativo e automático na família de modelos <code>o1/o3</code> .	Ativado via <code>"thinking"</code> ; devolvido num bloco <code>"thinking"</code> com <code>signature</code> .	Ativado via <code>thinkingConfig</code> ; suporta entrega de <code>thoughtSignature</code> nos <code>parts</code> .
Contabilização de <i>Tokens</i>	Objeto <code>usage</code> (campos <code>prompt_tokens</code> , <code>completion_tokens</code> , <code>total_tokens</code>).	Objeto <code>usage</code> (campos <code>input_tokens</code> , <code>output_tokens</code> ; sem <code>total</code> explícito).	Objeto <code>usageMetadata</code> (<code>promptTokenCount</code> , <code>candidatesTokenCount</code> , <code>totalTokenCount</code> , <code>thoughtsTokenCount</code>).
Identificador da Resposta	Campo <code>id</code> (e.g., <code>"chatcmpl-31"</code>) e metadados <code>object</code> , <code>created</code> .	Campo <code>id</code> (e.g., <code>"msg_01Xf..."</code>) e campo <code>type: "message"</code> .	Campo <code>responseId</code> e <code>modelVersion</code> ao nível raiz da resposta.

Tabela 2.2: Mapeamento técnico do processamento de respostas, execução e telemetria (*Responses*).

À semelhança dos pedidos, a estrutura JSON das respostas foi esquematizada para evidenciar as diferenças e semelhanças de cada resposta. A Figura 2.2 apresenta essa comparação estrutural. Os exemplos JSON completos das respostas encontram-se no Apêndice A.2.

OpenAI Response	Anthropic Response	Gemini Response
<pre>{ "id": "chatcmpl-...", "choices": [{ "index": 0, "message": { "role": "assistant", "content": "...", "tool_calls": [{ "id": "call_...", "type": "function", "function": { "name": "func", "arguments": "{ \"...\"}" } }] }, "finish_reason": "stop" }], "usage": { "prompt_tokens": 10, "completion_tokens": 20 } }</pre>	<pre>{ "id": "msg-...", "role": "assistant", "content": [{ "type": "text", "text": "..." }, { "type": "tool_use", "id": "toolu_...", "name": "func", "input": {...} }], "stop_reason": "end_turn", "usage": { "input_tokens": 10, "output_tokens": 20 } }</pre>	<pre>{ "candidates": [{ "content": { "role": "model", "parts": [{ "text": "..."}, { "functionCall": { "name": "func", "args": {...} } }] }, "finishReason": "STOP" }], "usageMetadata": { "promptTokenCount": 10, "candidatesTokenCount": 20 } }</pre>

Figura 2.2: Estruturas JSON esquemáticas e simplificadas de respostas para cada fornecedor.

A comparação das respostas reforça a percepção de que as três APIs divergem significativamente na organização estrutural, apesar de exprimirem os mesmos conceitos. A interface da OpenAI encapsula a resposta dentro de um *array* denominado **choices**, que contém um objeto **message** [1]; a interface da Anthropic expõe o conteúdo diretamente na raiz através de um *array content* com blocos tipados [2]; e a interface da Gemini utiliza **candidates** com **parts** e conceitos adicionais, como **thoughtSignature** [3].

Um aspeto particularmente relevante é a representação dos argumentos em chamadas de ferramenta: a OpenAI serializa-os como uma *string* JSON dentro do campo **arguments**, obrigando a uma desserialização adicional [1], enquanto a Anthropic e a Gemini devolvem JSON nativo nos campos **input** [2] e **args** [3], respetivamente. A contabilização de *tokens* também difere na nomenclatura e granularidade: a API da Gemini, por exemplo, expõe um campo adicional **thoughtsTokenCount** que permite distinguir os *tokens* de raciocínio dos *tokens* de conteúdo efetivo [3].

2.1.3 Divergências na Padronização de Esquemas de Validação

Para além das diferenças do corpo JSON, é importante referir a divergência na forma como cada fornecedor padroniza a tipagem de dados estruturados, nomeadamente na declaração de ferramentas (*tools*) e nas saídas estruturadas (*structured outputs*).

Enquanto a OpenAI [23] e a Anthropic [24] se apoiam nativamente na norma *JSON Schema* [25] para definir propriedades e parâmetros, a API da Gemini adota um subconjunto da especificação *OpenAPI 3.0* [26] (através do seu objeto *Schema* [27]). Isto reforça que a construção de uma camada de abstração exige não só o mapeamento de chaves, mas também a tradução e compatibilização entre diferentes definições de esquemas.

2.1.4 Conclusão do Estudo das APIs de Inferência

A realização deste estudo envolveu dificuldades práticas que merecem ser documentadas. Para a API da OpenAI, o acesso direto à API oficial implica custos de utilização, pelo que os testes foram realizados utilizando a API da Groq [14], que oferece um plano gratuito e implementa uma interface compatível com o formato da OpenAI (`/v1/chat/completions`). Adicionalmente, foram executados testes complementares contra uma instância local de Ollama [28], que também expõe um *endpoint* compatível com o formato OpenAI. Para a API da Anthropic, a situação foi semelhante: a API oficial requer um plano de subscrição pago, pelo que os testes foram realizados utilizando o Ollama, que disponibiliza um *endpoint* compatível com o formato da Anthropic (`/v1/messages`). Embora o Ollama implemente corretamente as interfaces de compatibilidade para pedidos simples, de *streaming* e com parâmetros de configuração, algumas funcionalidades mais avançadas, nomeadamente o *tool calling* estruturado e a geração de blocos de raciocínio (*thinking*) apresentaram limitações quando executadas em modelos de pequena dimensão, como o `llama3.2:1b`. Nesses casos, os exemplos de resposta foram ajustados para refletir com fidelidade o comportamento documentado das APIs oficiais. A API do Gemini foi a única cujos testes foram executados diretamente contra a API oficial, nomeadamente através do AI Studio [29], uma vez que este disponibiliza um nível de acesso gratuito.

Do ponto de vista técnico, o estudo permitiu concluir que as três APIs partilham um conjunto de conceitos fundamentais, como as mensagens, papéis, ferramentas, *streaming* e contabilização de *tokens*, mas implementam-nos de formas estruturalmente diferentes. As diferenças manifestam-se na nomenclatura dos campos, na organização hierárquica do JSON, na localização dos parâmetros de configuração e nos mecanismos de ativação de funcionalidades como *streaming* e raciocínio. Contudo, apesar destas similaridades conceituais, ainda existem campos proprietários em cada API de fornecedor de inferência. Esta análise permitiu perceber que uma abstração que pretenda cobrir estes três fornecedores precisa de ir além de uma simples tradução de nomes, exigindo uma compreensão das diferenças semânticas e estruturais observadas nos pedidos e respostas reais.

Conceito	Semelhança observada	Diferenças relevantes	Modelo comum pretendido
Modelo invocado	As três APIs precisam de identificar o modelo a usar.	OpenAI e Anthropic usam <code>model</code> no corpo; Gemini coloca o modelo no URI e, por vezes, também no cliente (SDK oficial).	<code>provider + model</code> , permitindo escolher adapter e preservar a identidade do modelo.
Histórico conversacional	Todas representam uma conversa com papéis e conteúdo.	OpenAI e Anthropic usam <code>messages</code> ; Gemini usa <code>contents</code> e <code>parts</code> ; Anthropic separa o <code>system</code> .	<code>CommonRequestMessage</code> , com papel comum e lista de partes de conteúdo.
Instruções de sistema	Todas suportam instruções globais para orientar o modelo.	OpenAI pode usar mensagem <code>system</code> ; Anthropic usa campo de topo <code>system</code> ; Gemini usa <code>system_instruction</code> .	<code>SystemPrompt</code> , separado do histórico para permitir tradução correta.
Parâmetros de geração	Temperatura, limite de tokens e opções de amostragem aparecem nos três ecossistemas.	Nomes e agrupamentos variam: campos soltos em OpenAI/Anthropic, <code>generationConfig</code> em Gemini, e opções exclusivas por fornecedor.	<code>CommonGenerationConfig</code> para o subconjunto comum e <code>providerOptions</code> para extensões.
Conteúdo multimodal e estruturado	As APIs evoluem para conteúdo composto, não apenas texto simples.	Anthropic e Gemini modelam blocos naturalmente; OpenAI depende do contrato usado; tool calling introduz partes específicas.	<code>RequestContentPart</code> e <code>ResponseContentPart</code> .
Utilização de tokens	Todas expõem algum tipo de contabilização de utilização.	OpenAI usa <code>prompt_tokens/ completion_tokens</code> ; Anthropic usa <code>input_tokens/ output_tokens</code> ; Gemini usa <code>usageMetadata</code> .	<code>CommonUsage</code> , com tokens de entrada, saída e total quando disponíveis.

Tabela 2.3: Mapeamento entre conceitos das APIs de inferência e o domínio comum

O caso de *tool calling* foi especialmente relevante, porque é onde as APIs mais divergem apesar de exprimirem uma intenção semelhante: permitir que o modelo peça a execução de uma função externa. Na OpenAI, a ferramenta é descrita em `tools[]` e a resposta devolve `tool_calls` com argumentos serializados como JSON em string. Na Anthropic, a ferramenta é descrita com `name`, `description` e `input_schema`; a resposta inclui blocos `tool_use`, e o resultado da execução regressa ao modelo como bloco `tool_result`. Na Gemini, as ferramentas são `functionDeclarations`, a chamada surge em `functionCall`, e o resultado é enviado como `functionResponse`.

Aspeto	API	Estrutura observada	Impacto no domínio comum
Declaração de ferramenta	OpenAI	<code>tools[].function.parameters.</code>	Normalização em <code>CommonTool.parametersSchema.</code>
Declaração de ferramenta	Anthropic	<code>tools[].input_schema.</code>	Mesmo conceito, nome de campo diferente.
Declaração de ferramenta	Gemini	<code>functionDeclarations[].parameters.</code>	Necessidade de traduzir declarações Gemini para a mesma estrutura comum.
Escolha de ferramenta	OpenAI	<code>tool_choice</code> com valores como <code>auto</code> , <code>none</code> ou ferramenta específica.	Representação em <code>ToolChoice.</code>
Escolha de ferramenta	Anthropic	<code>tool_choice</code> com semântica semelhante, incluindo <code>auto</code> , <code>any</code> e ferramenta específica.	O domínio preserva intenção, não o nome exato do campo.
Escolha de ferramenta	Gemini	<code>functionCallingConfig</code> dentro de <code>toolConfig.</code>	Conversão para a mesma decisão comum sobre uso de ferramenta.
Chamada pedida pelo modelo	OpenAI	<code>message.tool_calls[]</code> com <code>id</code> , <code>name</code> e <code>arguments.</code>	Mapeamento para <code>ToolCallPart.</code>
Chamada pedida pelo modelo	Anthropic	Bloco <code>tool_use</code> com <code>id</code> , <code>name</code> e <code>input.</code>	A chamada passa a ser uma parte estruturada da resposta.
Chamada pedida pelo modelo	Gemini	<code>functionCall</code> com <code>name</code> , <code>args</code> e, quando disponível, <code>id.</code>	O domínio não assume que todos os fornecedores expõem os mesmos identificadores.
Resultado de ferramenta	OpenAI	Mensagem com <code>role=tool</code> e <code>tool_call_id.</code>	Mapeamento para <code>ToolResultPart.</code>
Resultado de ferramenta	Anthropic	Bloco <code>tool_result.</code>	Resultado representado como conteúdo estruturado.
Resultado de ferramenta	Gemini	<code>functionResponse.</code>	Resultado traduzido para o mesmo conceito comum.
Eventos de <i>streaming</i>	Todas	OpenAI usa <code>delta</code> ; Anthropic usa eventos nomeados; Gemini envia respostas parciais por SSE.	Representação em <code>CommonResponseEvent</code> , separada de <code>CommonResponse.</code>

Tabela 2.4: Comparação do ciclo de *tool calling* e *streaming*

2.2 Model Context Protocol (MCP)

O *Model Context Protocol* (MCP) é um protocolo aberto criado para uniformizar a forma como aplicações de Inteligência Artificial acedem a ferramentas, dados e outras fontes de contexto [10]. Antes do surgimento do MCP, a integração destes recursos era normalmente realizada através de mecanismos específicos de cada aplicação ou fornecedor, resultando em soluções pouco reutilizáveis e difíceis de manter.

O protocolo define uma arquitetura cliente-servidor na qual os servidores disponibilizam capacidades que podem ser utilizadas por aplicações de IA. Estas capacidades são agrupadas em três categorias principais:

- **Tools:** operações que podem ser executadas pelo modelo, permitindo interagir com sistemas externos ou realizar ações específicas.
- **Resources:** informação disponibilizada pelos servidores para consulta, normalmente identificada através de URIs.
- **Prompts:** modelos de interação reutilizáveis que podem ser parametrizados e utilizados em diferentes cenários.

A comunicação entre clientes e servidores é realizada através de mensagens JSON-RPC e pode utilizar diferentes mecanismos de transporte, incluindo `stdio`, HTTP, SSE e WebSocket [30].

No modelo definido pelo protocolo, o *host* corresponde à aplicação responsável por coordenar as ligações aos servidores MCP. Dentro do *host*, os *clients* estabelecem ligações individuais com os servidores, que por sua vez disponibilizam as ferramentas, recursos e *prompts*. Esta separação permite que novas capacidades sejam adicionadas sem necessidade de alterar diretamente a lógica da aplicação que utiliza os modelos.

O estudo do MCP foi realizado em paralelo com a análise das APIs de inferência. Enquanto o objetivo dessa análise era compreender a interação com os modelos, o objetivo aqui foi perceber de que modo ferramentas e contexto externo podem ser disponibilizados de forma uniforme e independente do fornecedor do modelo.

Para complementar a análise teórica, foi desenvolvido um pequeno protótipo utilizando a biblioteca oficial `io.modelcontextprotocol:kotlin-sdk`. O protótipo teve como objetivo validar o ciclo básico de funcionamento do protocolo, incluindo a criação de um servidor, o registo de capacidades, o estabelecimento da ligação com um cliente e a troca de mensagens. Este exercício permitiu também verificar a compatibilidade da biblioteca com *Kotlin Multiplatform* [31].

O experimento realizado permitiu compreender na prática o funcionamento do protocolo e da biblioteca oficial. Foi possível validar o processo de descoberta de capacidades, a invocação de ferramentas e a troca de informação entre clientes e servidores MCP.

Este estudo permitiu também perceber que o acesso a ferramentas e fontes externas de contexto constitui uma preocupação distinta da comunicação com os modelos, tendo esta separação influenciado algumas decisões tomadas posteriormente na arquitetura da OmniAI.

Capítulo 3

Modelação do Domínio Comum

3.1 Visão Global do Domínio

No final do estudo prévio realizado no capítulo anterior, verificou-se uma acentuada fragmentação estrutural entre os principais fornecedores de Modelos de Linguagem de Grande Escala (LLMs) [13]. A ausência de uma padronização ou um protocolo comum leva à inconsistência entre as APIs da OpenAI [1], Anthropic [2] e Gemini [3], as quais têm campos distintos, diferentes ciclos de vida para o fluxo de dados e estruturas de objetos incompatíveis entre as várias APIs.

Para garantir a interoperabilidade, a arquitetura do OmniAI SDK assenta na criação de um modelo de domínio comum. A ideia foi criar um modelo de dados interoperável com os maiores fornecedores de inferência para representar pedidos, respostas e eventos dos mesmos, mas sem copiar diretamente a estrutura de nenhum deles.

Assim, o domínio funciona como um ponto intermédio estável que centraliza qualquer interação no OmniAI SDK. Isto significa que, independentemente de o cliente estruturar o seu pedido original no formato de um fornecedor específico (como por exemplo a Anthropic), esse contrato é imediatamente convertido para o domínio unificado e em seguida, para efetuar a chamada à API, o modelo comum é novamente traduzido para o formato nativo do fornecedor de destino. O mesmo processo inverso ocorre na receção da resposta, os dados que são devolvidos pela API externa são normalizados no nosso domínio comum antes de serem entregues ao cliente na estrutura esperada. Este fluxo completo de transformações está representado visualmente na Figura 3.1.

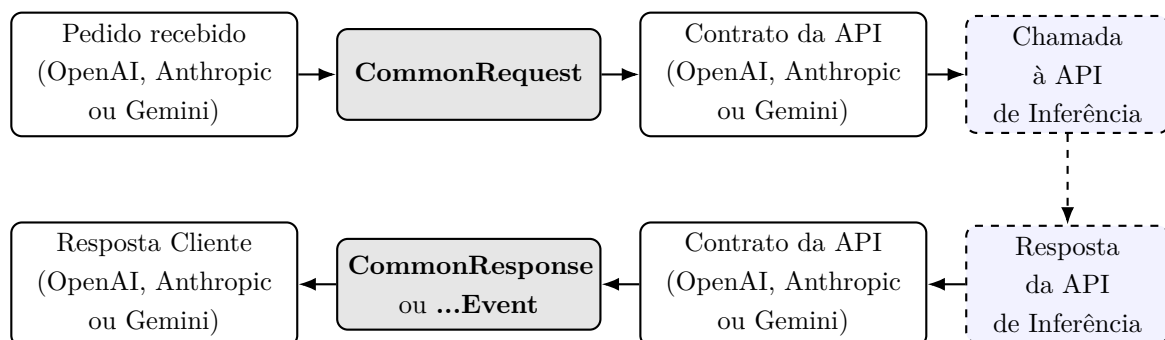


Figura 3.1: Fluxo de normalização entre contratos externos e domínio comum (OmniAi-SDK)

3.2 Estrutura do Domínio Comum

O domínio comum do **OmniAI SDK** que consite no conjunto de classes e relações que padroniza as interações com os diferentes fornecedores divide-se em dois blocos principais: **Request** e **Response**.

- O **Request** define a estrutura dos pedidos enviados ao modelo.
- O **Response** descreve as respostas devolvidas, quer no formato (**CommonResponse**), quer em formato incremental através de eventos (**CommonResponseEvent**).

3.2.1 Request

O **CommonRequest** é o modelo base que normaliza qualquer pedido a LLMs, garantindo um contrato único e consistente, independentemente do fornecedor. A estrutura foi desenhada para manter os elementos essenciais de geração (fornecedor, modelo e mensagens) e, ao mesmo tempo, permitir extensões controladas via configuração e opções específicas do provider. Como se observa na Figura 3.2, o **CommonRequest** agrega diretamente o **provider**, o **model** e a lista de **messages**, que representam o histórico conversacional. Cada **CommonRequestMessage** contém o papel (**CommonRole**) e o conteúdo em partes (**RequestContentPart**), permitindo combinar texto, JSON ou chamadas a ferramentas. A configuração de geração é isolada em **CommonGenerationConfig**, enquanto as ferramentas disponíveis são descritas por **CommonTool** e controladas por **ToolChoice**. Por fim, **SystemPrompt** e **providerOptions** suportam personalizações globais e específicas do fornecedor, mantendo o núcleo estável e extensível.

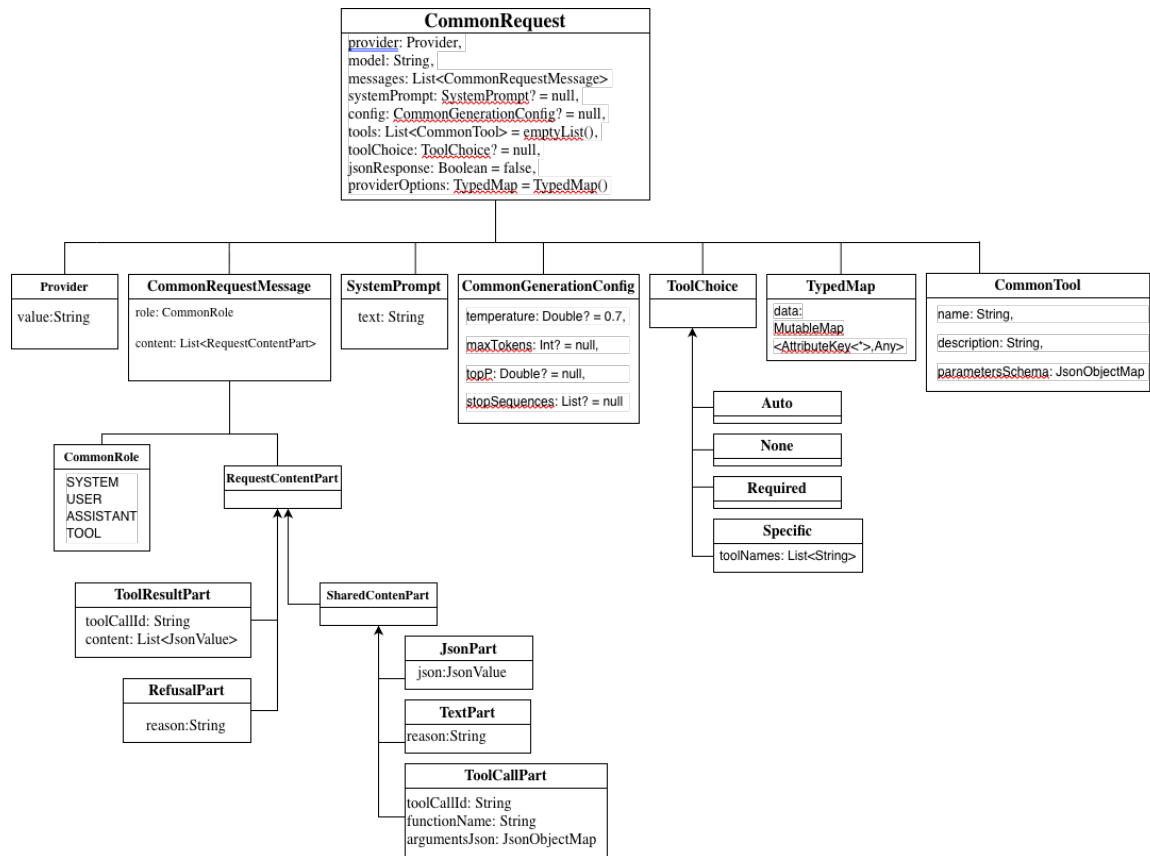


Figura 3.2: Estrutura de classes do CommonRequest.

Para sistematizar a estrutura apresentada na Figura 3.2, a Tabela 3.1 agrega todos os parâmetros relevantes do domínio de *Request*, incluindo o objeto principal e os tipos diretamente associados. O objetivo é apresentar o significado funcional de cada campo no fluxo de construção do pedido.

Elemento	Campo	Tipo	Descrição
CommonRequest	provider	Provider	Fornecedor alvo da chamada.
CommonRequest	model	String	Identificador do modelo a utilizar.
CommonRequest	messages	List<Common-RequestMessage>	Histórico de mensagens enviado ao modelo.
CommonRequest	systemPrompt	SystemPrompt?	Prompt de sistema global (opcional).
CommonRequest	config	Common-GenerationConfig?	Parâmetros de geração (opcional).
CommonRequest	tools	List<CommonTool>	Ferramentas disponíveis para chamadas de função.
CommonRequest	toolChoice	ToolChoice?	Política de seleção de ferramentas.
CommonRequest	jsonResponse	Boolean	Indica se a resposta deve ser forçada a JSON.
CommonRequest	providerOptions	TypedMap	Opções específicas do fornecedor.
CommonRequest-Message	role	CommonRole	Papel da mensagem (SYSTEM, USER, ASSISTANT, TOOL).
CommonRequest-Message	content	List<Request-ContentPart>	Conteúdo segmentado da mensagem.
SystemPrompt	text	String	Conteúdo do prompt de sistema.
Common-GenerationConfig	temperature	Double?	Grau de aleatoriedade da geração.
Common-GenerationConfig	maxTokens	Int?	Limite máximo de tokens de saída.
Common-GenerationConfig	topP	Double?	Probabilidade cumulativa (nucleus sampling).
Common-GenerationConfig	stopSequences	List<String>?	Sequências que interrompem a geração.
CommonTool	name	String	Nome da ferramenta.
CommonTool	description	String	Descrição funcional da ferramenta.
CommonTool	parametersSchema	JsonObjectMap	Esquema JSON dos parâmetros.
ToolChoice	Auto / None / Required / Specific	–	Estratégia de uso de ferramentas (auto, nenhuma, obrigatório ou lista específica).
ToolChoiceAuto	–	–	Subtipo sem campos (seleção automática).
ToolChoiceNone	–	–	Subtipo sem campos (sem ferramentas).
ToolChoiceRequired	–	–	Subtipo sem campos (uso obrigatório).
ToolChoiceSpecific	toolNames	List<String>	Lista de ferramentas permitidas.
Provider	value	String	Identificador textual do fornecedor.
TypedMap	data	Map<String, Any?>	Mapa genérico de opções específicas do fornecedor.
RequestContentPart	TextPart	text: String	Conteúdo textual.
RequestContentPart	JsonPart	json: JsonValue	Conteúdo JSON estruturado.
RequestContentPart	ToolCallPart	toolCallId, functionName, argumentsJson	Chamada a ferramenta com argumentos.
RequestContentPart	ToolResultPart	toolCallId, content	Resultado de execução de ferramenta.

Tabela 3.1: Parâmetros do domínio de *Request*.

3.2.2 Response

O **CommonResponse** é o modelo base que normaliza as respostas devolvidas pelos diferentes fornecedores, garantindo um contrato único e consistente para o cliente. A estrutura foi desenhada para representar o resultado final da geração, mantendo as escolhas (**CommonChoice**) com o

respetivo índice e a razão de término (**FinishReason**). Cada escolha contém uma mensagem (**CommonResponseMessage**) com o papel (**CommonRole**) e o conteúdo em partes (**ResponseContentPart**), permitindo combinar texto, JSON, chamadas a ferramentas ou recusas. A informação de uso (**CommonUsage**) isola o consumo de tokens e facilita métricas e contabilização. Por fim, **providerOptions** preserva metadados específicos do fornecedor, mantendo o núcleo do domínio estável e extensível.

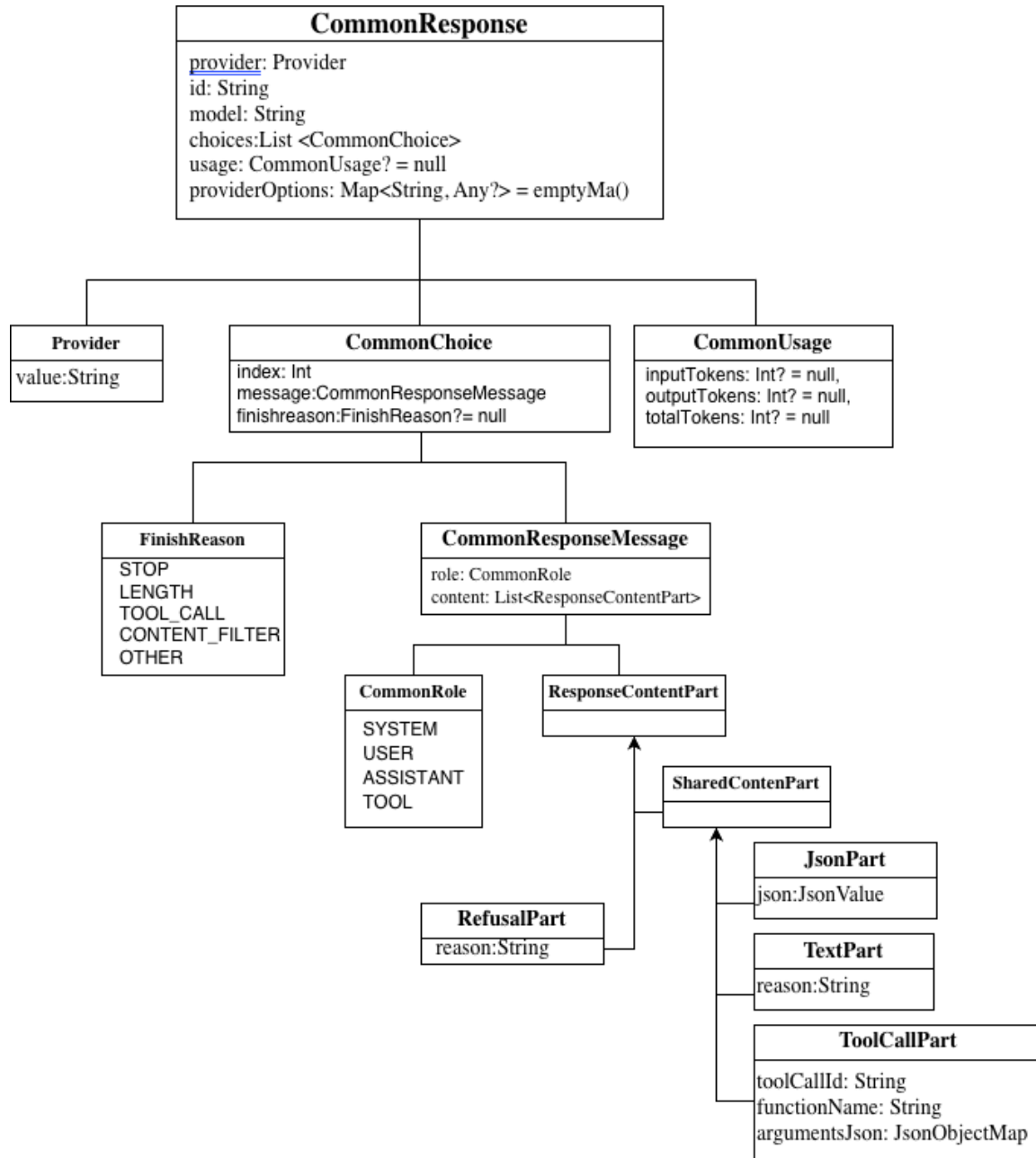


Figura 3.3: Estrutura de classes do **CommonResponse**.

A Tabela 3.2 reúne os campos do domínio de *Response*, incluindo o objeto base, as escolhas e os eventos de streaming.

Elemento	Campo	Tipo	Descrição
CommonResponse	provider	Provider	Fornecedor que produziu a resposta.
CommonResponse	id	String	Identificador do pedido/execução.
CommonResponse	model	String	Identificador do modelo utilizado.
CommonResponse	choices	List<CommonChoice>	Lista de escolhas geradas.
CommonResponse	usage	CommonUsage?	Contabilização de tokens (opcional).
CommonResponse	providerOptions	Map<String, Any?>	Metadados específicos do fornecedor.
CommonChoice	index	Int	Índice da escolha.
CommonChoice	message	CommonResponseMessage	Mensagem com papel e conteúdo.
CommonChoice	finishReason	FinishReason?	Razão de término da escolha (opcional).
CommonResponseMessage	role	CommonRole	Papel da mensagem.
CommonResponseMessage	content	List<ResponseContentPart>	Conteúdo segmentado da resposta.
CommonUsage	inputTokens	Int?	Tokens de entrada.
CommonUsage	outputTokens	Int?	Tokens de saída.
CommonUsage	totalTokens	Int?	Total de tokens.
FinishReason	–	STOP / LENGTH / TOOL_CALL / CONTENT_FILTER / OTHER	Motivo de conclusão da geração.
ResponseContentPart	TextPart	text: String	Segmento textual.
ResponseContentPart	JsonPart	json: JsonValue	Segmento JSON estruturado.
ResponseContentPart	ToolCallPart	toolCallId, functionName, argumentsJson	Chamada a ferramenta.
ResponseContentPart	RefusalPart	reason: String	Recusa do modelo com justificção.

Tabela 3.2: Parâmetros do domínio de *Response* (CommonResponse).

O **CommonResponseEvent** complementa o **CommonResponse** ao representar respostas em streaming como uma sequência ordenada de eventos. Esta abordagem permite construir a saída de forma incremental: primeiro sinaliza o início da resposta, depois o arranque de cada escolha, segue com deltas de texto ou de argumentos de ferramentas e, por fim, reporta o fecho da escolha, o uso de tokens e a conclusão (ou erro) do stream. Cada evento partilha um conjunto mínimo de metadados (fornecedor, identificador, modelo e sequência) que asseguram consistência e reordenação determinística.

Elemento	Campo	Tipo	Descrição
CommonResponse Event	provider	Provider	Fornecedor associado ao evento.
CommonResponse Event	id	String	Identificador do pedido/execução.
CommonResponse Event	model	Model	Modelo utilizado no evento.
CommonResponse Event	sequence	Long	Ordem do evento no stream.
CommonResponse Event	provider EventType	String?	Tipo bruto do evento do fornecedor (opcional).
ResponseStarted	–	–	Início do streaming da resposta.
ChoiceStarted	choiceIndex	Int	Índice da escolha iniciada.
ChoiceStarted	role	CommonRole?	Papel associado à escolha (opcional).
TextDeltaEvent	choiceIndex	Int	Índice da escolha que recebe o delta.
TextDeltaEvent	text	String	Fragmento incremental de texto.
ToolCall StartedEvent	choiceIndex	Int	Índice da escolha.
ToolCall StartedEvent	toolCallIndex	Int	Índice da chamada a ferramenta.
ToolCall StartedEvent	toolCallId	String	Identificador da chamada a ferramenta.
ToolCall StartedEvent	functionName	String	Nome da função invocada.
ToolCallArguments DeltaEvent	choiceIndex	Int	Índice da escolha.
ToolCallArguments DeltaEvent	toolCallIndex	Int	Índice da chamada a ferramenta.
ToolCallArguments DeltaEvent	arguments Fragment	String	Fragmento incremental de argumentos.
ChoiceFinished	choiceIndex	Int	Índice da escolha terminada.
ChoiceFinished	finishReason	FinishReason?	Razão de término (opcional).
UsageReported	usage	CommonUsage	Contabilização de tokens reportada.
ResponseCompleted	–	–	Conclusão do streaming da resposta.
ResponseErrored	message	String	Mensagem de erro.
ResponseErrored	retryable	Boolean	Indica se o erro é recuperável.

Tabela 3.3: Parâmetros do domínio de *Response* (CommonResponseEvent).

3.3 TypedMap e Extensibilidade do Domínio

Nem todos os dados relevantes pertencem ao modelo mínimo de inferência. Algumas informações são específicas do fornecedor, como parâmetros avançados do Gemini e outras pertencem ao contexto operacional, como metadados de autenticação, IP do cliente, cabeçalhos HTTP ou atributos necessários para observabilidade e autorização. Para estes casos, o **OmniAI SDK** introduz a estrutura **TypedMap**.

O **TypedMap** funciona como um mapa de atributos tipado por chave. Cada **AttributeKey<T>** associa um nome lógico a um tipo esperado, permitindo guardar e recuperar valores sem depender de conversões frágeis de strings. Esta abordagem é mais segura do que um **Map<String, Any?>**

puro, porque cada chave transporta também a expectativa de tipo. Ao mesmo tempo, é mais flexível do que adicionar campos ao domínio sempre que surge uma necessidade nova.

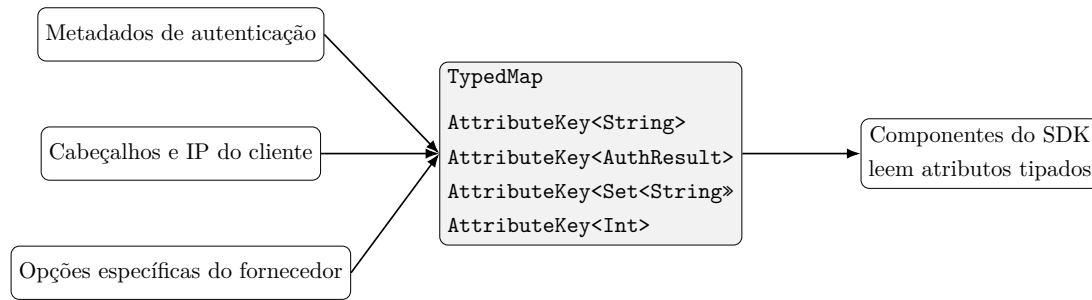


Figura 3.4: Utilização da `TypedMap` para transportar atributos extensíveis

No domínio de inferência, a `TypedMap` aparece em `providerOptions`, permitindo preservar dados que não pertencem ao subconjunto comum mas não devem ser perdidos durante a tradução. Fora da inferência estrita, a mesma estrutura permite transportar metadados entre componentes sem criar dependências diretas entre eles. Esta característica é útil para a evolução futura do **OmniAI SDK**, porque permite que mecanismos ,como autorização, observabilidade ou políticas de execução, usem atributos adicionais sem forçar alterações constantes nas classes centrais do domínio.

Capítulo 4

Arquitetura e Desenho do Sistema

A arquitetura de alto nível (*top-level*) apresentada na Figura 4.1 posiciona o **OmniAI SDK** como o principal resultado deste projeto. Em vez de constituir uma aplicação final isolada, o SDK fornece um conjunto de abstrações, componentes e mecanismos reutilizáveis que permitem construir soluções de integração com múltiplos fornecedores de modelos de linguagem de forma consistente. Entre estes componentes encontram-se o domínio comum de pedidos e respostas, os contratos normalizados, os adaptadores responsáveis pela tradução entre o modelo interno e as APIs externas, o *pipeline* de interceptação para funcionalidades transversais e uma DSL de configuração que simplifica a composição do sistema.

A utilização do SDK traduz-se, na prática, na criação de uma **OmniAI Gateway** adaptada às necessidades de cada utilizador. A Gateway funciona como uma camada de acesso unificada sobre diferentes fornecedores de modelos, abstraindo as diferenças existentes entre APIs e disponibilizando uma interface homogênea para clientes e aplicações consumidoras. Desta forma, aspetos relacionados com autenticação, observabilidade, resiliência, encaminhamento de pedidos ou integração com ferramentas externas podem ser configurados ao nível da Gateway sem impactar a lógica das aplicações que a utilizam.

Para validar a arquitetura proposta e demonstrar a aplicabilidade do SDK num cenário real, foi desenvolvida uma instância concreta da **OmniAI Gateway** como Prova de Conceito (*Proof of Concept* – PoC). Esta instância é construída integralmente através dos mecanismos disponibilizados pelo SDK, recorrendo a configurações declarativas para definir fornecedores, modelos, políticas de autenticação, métricas e restantes componentes da infraestrutura. Durante o arranque da aplicação, estas configurações são utilizadas para compor dinamicamente os adaptadores, a *pipeline* de processamento e os conectores HTTP baseados em Ktor, expondo posteriormente um conjunto de *endpoints* compatíveis com os contratos normalizados definidos pelo domínio comum.

Esta separação entre SDK e Gateway constitui uma das principais decisões arquiteturais do projeto. O SDK concentra toda a lógica reutilizável e independente do contexto de execução, enquanto cada Gateway representa uma instanciação específica dessa infraestrutura, podendo ser configurada para diferentes fornecedores, requisitos operacionais ou cenários de utilização sem necessidade de alterar os componentes centrais do sistema.

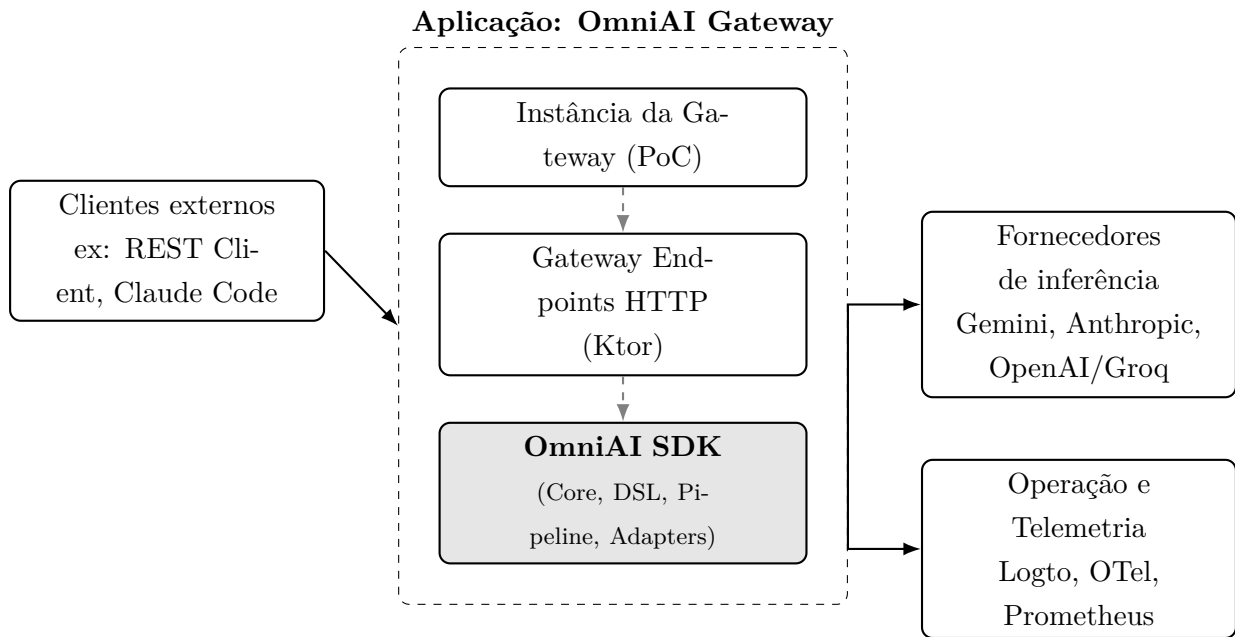


Figura 4.1: Arquitetura *top-level*: O OmniAI SDK utilizado para instanciar uma OmniAI Gateway (configurada como PoC).

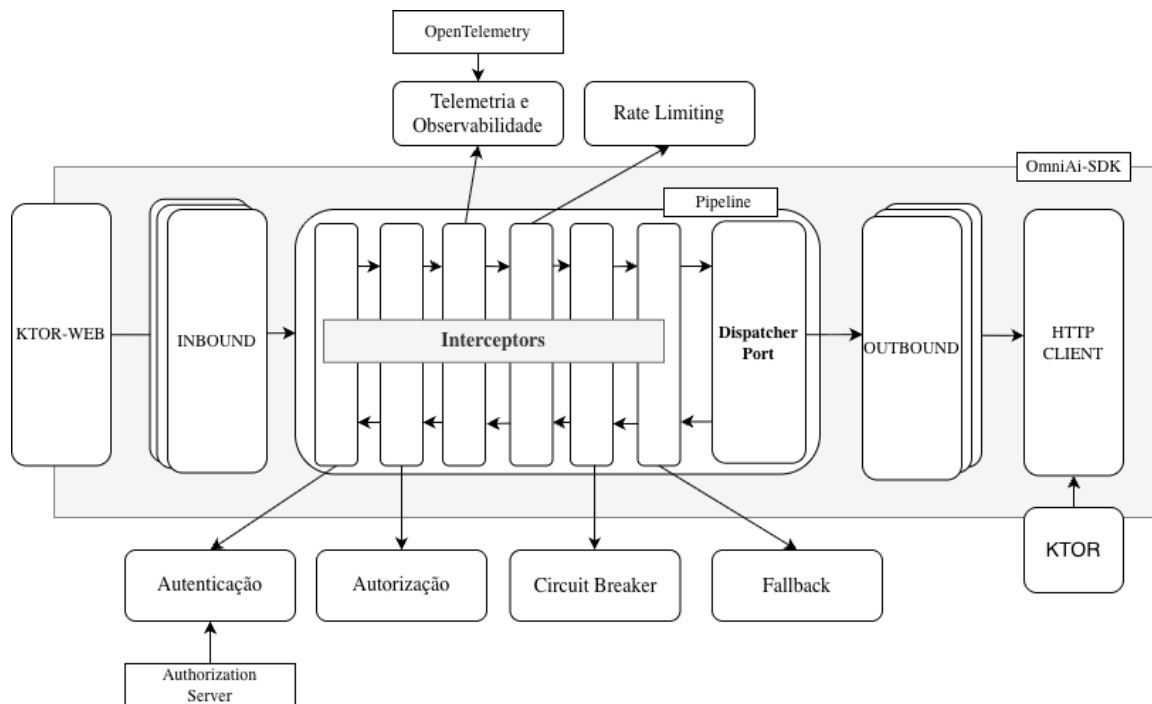


Figura 4.2: Visão da integração entre Ktor, inbounds, pipeline, interceptors, outbounds e HTTP client no OmniAI SDK

A Figura 4.1 mostra que a **OmniAI Gateway** não contém a lógica essencial de tradução, despacho ou resiliência. Essa lógica pertence ao **OmniAI SDK** e é apenas instanciada pela aplicação final. A Figura 4.2 aproxima a visão interna: um conector HTTP recebe o corpo do

pedido, converte-o para um DTO de contrato, invoca um *inbound adapter*, e este chama um *DispatcherPort*. A partir desse ponto, a execução pode seguir por um dispatcher direto ou por um dispatcher apoiado numa *pipeline*. No final, um *outbound adapter* traduz o pedido comum para o contrato do fornecedor e utiliza um cliente HTTP abstrato para executar a chamada real.

Esta organização permite que aplicações já compatíveis com APIs existentes possam ser integradas com esforço reduzido. Por exemplo, uma aplicação preparada para a API Anthropic pode alterar o seu `baseURL` para apontar para a OmniAI Gateway, mantendo o contrato esperado pelo cliente. O mesmo princípio aplica-se a clientes compatíveis com a api de inferencia da OpenAI e api de inferencia do Gemini.

A arquitetura foi também pensada para escalamento horizontal: o SDK evita estado global obrigatório, concentra estado transitório no contexto do pedido e define portas para substituir stores em memória por backends partilhados quando a Gateway é executada em múltiplas instâncias.

4.1 Arquitetura Hexagonal (Ports & Adapters)

A Arquitetura Hexagonal, também conhecida como padrão de Portas e Adaptadores, foi escolhida neste projeto para garantir que a lógica principal do sistema permaneça independente de detalhes técnicos e de integrações externas. Em vez de o *core* da aplicação depender diretamente de bibliotecas, protocolos ou APIs específicas, toda a comunicação com o exterior acontece através de interfaces bem definidas, enquanto os adaptadores tratam das implementações concretas [7, 32, 33].

Esta separação torna-se especialmente importante no contexto de uma *Gateway* de Inteligência Artificial, uma vez que o sistema atua como ponto central de comunicação entre clientes HTTP, fornecedores de inferência, mecanismos de autenticação e serviços de observabilidade. Sem um isolamento arquitetural claro, a lógica de domínio acabaria inevitavelmente dependente de *SDKs* externos, formatos proprietários e detalhes específicos de integração de cada fornecedor.

Ao adotar esta abordagem, foi possível manter o *core* do OmniAI *SDK* focado apenas nas responsabilidades centrais de normalização, abstração e coordenação. Assim, funcionalidades como chamadas de rede, serialização de dados ou integração com APIs externas ficam confinadas aos adaptadores. Isto permite que novos fornecedores de IA ou novos mecanismos de autenticação sejam adicionados com alterações mínimas na lógica principal da aplicação.

Durante o desenho da solução, foram analisadas outras abordagens arquiteturais para avaliar qual delas se ajustava melhor aos requisitos do projeto. Uma das alternativas consideradas foi a arquitetura tradicional em camadas. Apesar de ser uma solução mais simples numa fase inicial, este modelo tende a perder clareza estrutural à medida que o sistema cresce. Com o tempo, torna-se comum encontrar regras de negócio misturadas com lógica de transporte, chamadas HTTP e tratamento de contratos externos, aumentando o acoplamento e dificultando a manutenção [34].

Neste contexto, a Arquitetura Hexagonal revelou-se a solução mais adequada para os objetivos do projeto. A abordagem oferece modularidade, isolamento e flexibilidade suficientes para lidar com as diferentes políticas transversais exigidas por uma *Gateway* de Inteligência Artificial, como autenticação, resiliência, observabilidade e roteamento de pedidos, mantendo ao mesmo

tempo uma estrutura organizada e simples de evoluir [17–19]. Além disso, esta organização arquitetural facilita a introdução de novos fornecedores e novas funcionalidades sem comprometer a estabilidade do núcleo da aplicação.

4.1.1 Contracts

Os módulos `contracts:openai`, `contracts:anthropic` e `contracts:gemin`i modelam os formatos de pedido, resposta e eventos de cada fornecedor. Estes contratos são DTOs no sentido em que representam dados transportados na fronteira do sistema, mas não são apenas DTOs genéricos. São chamados *contracts* porque codificam uma compatibilidade externa concreta: nomes de campos, estruturas JSON, eventos de *streaming*, campos opcionais, formatos de erro e convenções específicas de cada API.

Esta distinção é importante. Um DTO interno poderia ser livremente alterado sem impacto externo; um contrato compatível com OpenAI, Anthropic ou Gemini precisa de respeitar o formato que clientes e fornecedores esperam. Ao isolar estes contratos em módulos próprios, o SDK evita que detalhes como `tool_calls`, `content_block_delta` ou `functionDeclarations` se espalhem pelo domínio comum.

4.1.2 Inbounds

Os *inbound adapters* recebem pedidos no formato do cliente e traduzem-nos para `CommonRequest`. Cada inbound implementa a porta `InboundPort` e recebe um `DispatcherPort`, não uma implementação concreta de pipeline ou de fornecedor. Além disso, cada inbound usa uma implementação de `InboundTranslator`, responsável por converter o objeto de entrada para domínio e por converter a resposta comum de volta para o contrato do cliente.

Esta decisão evita que o inbound exponha diretamente uma Web API. O SDK fornece adapters que trabalham sobre objetos de transferência, e a exposição HTTP fica noutra adapter, como o `gateway-ktor-server`. Assim, um servidor Ktor, uma função serverless, um handler Express ou outro mecanismo de entrada podem chamar o mesmo inbound desde que consigam construir o DTO esperado. Esta separação reduz a dependência do domínio e dos inbounds face a bibliotecas HTTP externas.

4.1.3 Outbounds

Os *outbound adapters* fazem o caminho inverso: recebem um `CommonRequest`, traduzem-no para o contrato do fornecedor real e executam a chamada externa. Cada *outbound* implementa `OutboundPort`, identifica `provider`, `model` e `key`, e delega a tradução num `OutboundTranslator`. A implementação atual inclui *outbounds* para OpenAI, Anthropic e Gemini.

Ao contrário dos *inbounds*, os *outbounds* precisam de comunicar com serviços externos. Por isso, recebem uma implementação de `HttpTransportClient`, a porta de transporte HTTP descrita na Secção 4.1.4, que abstrai por completo o mecanismo de rede utilizado. Esta inversão de dependência é o que permite testar *adapters* sem depender de chamadas reais e evoluir o ambiente de execução sem reescrever o domínio.

4.1.4 `HttpTransportClient`

A interface `HttpTransportClient`, definida no módulo `core`, representa a porta de saída responsável por toda a comunicação HTTP do *SDK* com os fornecedores externos. Esta abstração isola completamente os *outbound adapters* de qualquer biblioteca ou mecanismo concreto de rede, seguindo o princípio de inversão de dependência que caracteriza a Arquitetura Hexagonal.

A interface expõe três operações distintas. A operação `execute` é utilizada para pedidos síncronos: recebe uma configuração tipificada de pedido (`RequestConfig`) e um serializador, executa a chamada HTTP e devolve um `HttpCallResult`. A operação `listen` destina-se a respostas em *streaming* baseadas em *Server-Sent Events* (SSE) com um único tipo de evento, devolvendo um `Flow` de resultados. A operação `listenMany` generaliza o modelo de SSE para cenários com múltiplos tipos de eventos, selecionando o serializador adequado com base no campo `event` de cada mensagem.

O resultado de qualquer chamada HTTP é representado pela classe selada `HttpCallResult`, parametrizada pelo tipo da resposta esperada. Esta classe cobre os seguintes casos: `Success`, que encapsula os dados desserializados; `ApiError`, que retém o código de estado HTTP e o corpo da resposta de erro; `NetworkError`, originado por falhas de conectividade; `SerializationError`, resultante de respostas com formato inesperado; e `UnknownError`, para situações não classificadas. Todos os casos carregam um `TypedMap` de metadados, que permite propagar cabeçalhos de resposta e outras informações contextuais para os adaptadores e intercetores de forma estruturada.

A implementação concreta fornecida pelo *SDK*, `KtorHttpTransportClient`, assenta no cliente HTTP Ktor e reside no módulo `contracts:ktor-http`. Esta implementação suporta re-tentativas configuráveis com atraso entre tentativas, tratamento explícito de exceções de rede e de serialização, e integração com SSE nativo do Ktor. A configuração do cliente HTTP subjacente é realizada através de `PlatformHttpClient`, que inclui *timeouts* de ligação e de *socket*, suporte a *Content Negotiation* e o *plugin* SSE. Esta configuração é fornecida por implementações específicas de plataforma neste caso JVM e JavaScript, permitindo ao *SDK* operar como uma biblioteca *Kotlin Multiplatform*.

A separação entre a interface `HttpTransportClient` e a sua implementação em Ktor tem implicações práticas relevantes. Em contextos de teste, os *outbounds* podem ser exercitados com um cliente fictício sem realizar chamadas reais. Em contextos de produção, é possível substituir a implementação padrão por uma versão instrumentada com métricas, por um cliente com configurações específicas de *proxy* ou por uma implementação adaptada a um ambiente de execução distinto, sem alterar qualquer lógica de adaptação ou de domínio.

4.1.5 `Dispatcher`

O `DispatcherPort` é a porta chamada pelos *inbounds* para executar um `CommonRequest`. A sua responsabilidade não é, por si só, conter inteligência de roteamento; é despachar o pedido para uma estratégia de execução. No *SDK* é fornecida uma implementação simples, criada por `routingDispatcherFactory`, que procura o *outbound* cujo `provider` e `model` correspondem aos campos do pedido e envia a chamada para esse adapter.

Como se trata de uma interface, a arquitetura permite outras implementações com objetivos

diferentes. Um consumidor do SDK pode fornecer um dispatcher direto, um dispatcher com seleção por custo, um dispatcher que consulta políticas externas ou um dispatcher que aplica regras mais complexas. A *pipeline* fornecida pelo SDK é ela própria encaixada através de um adapter, `PipelineBackedDispatcher`, preservando a ideia de que os inbounds dependem apenas de `DispatcherPort`.

4.2 Ecossistema do SDK

O SDK está organizado como um projeto Gradle multi-módulo. Os módulos principais incluem `core`, `contracts`, `inbound`, `outbound`, `interceptors`, `mcp-broker`, `dispatcher`, `gateway-client`, `gateway-ktor-server` e `contracts:ktor-http`. Esta divisão permite evoluir cada área com responsabilidades claras: domínio e portas em `core`, contratos externos em `contracts`, adapters de entrada e saída em módulos próprios, preocupações transversais em `interceptors`, montagem declarativa em `gateway-client` e exposição HTTP/Ktor em `gateway-ktor-server`.

O módulo `contracts:ktor-http` fornece a abstração de transporte `HttpTransportClient` e a implementação `KtorHttpTransportClient`. Esta camada encapsula chamadas HTTP, SSE, serialização, metadados de resposta, erros de rede e erros de parsing. Ao colocá-la atrás de uma interface, os outbounds não ficam presos a uma implementação específica de cliente HTTP.

O módulo `gateway-client` disponibiliza uma DSL para montar uma Gateway. A aplicação consumidora pode declarar outbounds, interceptors, autenticação, autorização e modo de execução. A DSL permite escolher entre `useNativePipeline`, que monta a pipeline e o dispatcher padrão, e `useCustomDispatcher`, que permite substituir totalmente a estratégia de execução. Esta flexibilidade é essencial para um SDK: o POC usa a montagem nativa, mas outro consumidor pode querer um dispatcher próprio.

A instância da OmniAI Gateway desenvolvida no âmbito deste projeto foi concebida exclusivamente como uma Prova de Conceito (*Proof of Concept* - PoC), com o objetivo de validar o processo de instanciação e configuração disponibilizado pelo SDK. Durante o desenvolvimento, a solução foi estruturada recorrendo à funcionalidade de *composite build* do Gradle, através da diretiva `includeBuild("../OmniAi-SDK")` [35].

Esta decisão não pretende representar o modelo final de utilização da plataforma em ambiente de produção. A sua adoção teve um propósito essencialmente prático, permitindo desenvolver e compilar simultaneamente o SDK e a Gateway instanciada sem depender de ciclos constantes de publicação de versões intermédias da biblioteca. Desta forma, qualquer alteração efetuada no núcleo do SDK podia ser imediatamente refletida e validada no PoC, reduzindo significativamente o tempo necessário para testar novas funcionalidades, ajustes arquiteturais ou alterações ao pipeline interno da Gateway.

Sem esta abordagem, o processo de desenvolvimento tornar-se-ia substancialmente mais lento, uma vez que cada alteração implicaria publicar uma nova versão da biblioteca, atualizar dependências no projeto consumidor e recompilar novamente toda a aplicação. A utilização de *composite builds* permitiu assim criar um ciclo de desenvolvimento mais rápido e iterativo, particularmente importante numa fase de exploração arquitetural e validação técnica do SDK.

Num cenário de adoção real, a expectativa arquitetural é diferente. O SDK deverá ser consumido como uma dependência externa devidamente versionada, distribuída através de pla-

taformas de gestão de dependências, como Maven no ecossistema JVM ou NPM em ambientes Node.js e TypeScript. Desta forma, diferentes equipes poderão instanciar e integrar a sua própria Gateway de forma independente, mantendo um processo de versionamento e distribuição consistente com as práticas tradicionais de engenharia de software.

Capítulo 5

Implementação e Evolução Técnica

5.1 Stack Tecnológica e Metodologias

O **OmniAI SDK** foi desenvolvido em Kotlin Multiplatform [6] com targets JVM e JavaScript/Node.js. A escolha de KMP permite partilhar o domínio, os contratos, as portas, os adapters principais e parte da lógica geral entre plataformas, mantendo implementações específicas quando o ambiente o exige [22]. A serialização dos contratos é baseada em Kotlin Serialization [36], o que facilita a definição explícita dos modelos JSON estudados.

O Ktor [37] é usado em dois pontos diferentes, que importa distinguir. Primeiro, o **OmniAI SDK** fornece uma implementação HTTP baseada em Ktor, através de `KtorHttpClient`, usada pelos outbounds para executar pedidos HTTP e SSE. Segundo, o **OmniAI SDK** fornece o módulo `gateway-ktor-server`, que disponibiliza conectores Ktor para expor rotas compatíveis com OpenAI, Anthropic e Gemini. Assim, convém clarificar que o OmniAI SDK não possui qualquer dependência do Ktor, sendo este último apenas uma implementação concreta adotada pelo nosso Proof of Concept **OmniAI Gateway** que consiste em uma aplicação Ktor independente, construída como prova de conceito para demonstrar o uso do OmniAI SDK num sistema real.”

As ferramentas de desenvolvimento incluíram Git, IntelliJ IDEA, o cliente HTTP do IDE para pedidos REST, Docker Compose para Logto [38], PostgreSQL [39], OpenTelemetry Collector [40], Prometheus [41] e Grafana [42], além de clientes reais como Claude Code e SDKs oficiais dos fornecedores. Também foram usados assistentes de apoio, como Gemini no browser e agentes de programação, para pesquisa, comparação de documentação e aceleração de tarefas, mantendo a validação no código, nos testes e nos cenários executados. A pipeline de CI [43] existente executa `./gradlew clean build` como passo evolutivo, está prevista a criação de uma pipeline de CD para publicar as versões do nosso Software Development Kit, o **OmniAI SDK** em Maven e NPM.

5.2 A Primeira Iteração da Gateway

A primeira iteração do projeto consistiu numa Gateway HTTP construída diretamente, responsável por receber pedidos de clientes e encaminhá-los para fornecedores de inferência. Esta fase foi útil para validar rapidamente a viabilidade da solução, testar chamadas reais e compreen-

der diferenças práticas entre as diferentes API's dos fornecedores, neste caso da OpenAI [1], Anthropic [2] e Gemini [3].

À medida que o sistema cresceu, essa abordagem revelou acoplamento excessivo. A tradução entre contratos, a configuração dos fornecedores, a autenticação, o tratamento de erros e o encaminhamento de pedidos começaram a concentrar-se na própria aplicação. Cada novo fornecedor ou preocupação exigia alterações no mesmo conjunto de componentes, tornando o código difícil de evoluir e de se testar.

A reorganização separou o produto principal, o **OmniAI SDK**, da aplicação demonstradora, a **OmniAI Gateway**. O OmniAI SDK passou a concentrar o domínio comum, contratos, adapters, portas, *dispatcher*, *pipeline*, *interceptors*, DSL e abstrações HTTP. A **OmniAI Gateway** passou a ser um Proof Of Concept executável, usado para demonstrar o uso do **OmniAI SDK** num sistema real, este carrega configuração, instancia os componentes necessários, instala os conectores Ktor e expõe endpoints HTTP. Esta aplicação não reimplementa a lógica central da biblioteca, ela valida que essa lógica pode ser composta, configurada e executada fora dos módulos internos do **OmniAI SDK**.

Esta separação também influenciou o desenho para escalamento horizontal. O **OmniAI SDK** foi projetado para manter o estado do pedido dentro de estruturas transitórias, como `GatewayContext` e `TypedMap`, e para expor portas onde existe estado que é partilhado, como caches, rate limiting, circuit breaker, autenticação ou observabilidade. A arquitetura permite que uma implementação inicial utilize armazenamento em memória para simplificar o Proof Of Concept mas, por outro lado, em um cenário de escalamento horizontal, esses mecanismos podem ser substituídos por backends partilhados, sem implicar qualquer alteração nos inbounds, outbounds ou contratos.

5.3 Decisões de Desenho e Alternativas Descartadas

A análise de *trade-offs* partiu de três critérios: proteger a estabilidade da versão demonstrável, manter o **OmniAI SDK** reutilizável fora do *Proof of Concept* e evitar que funcionalidades experimentais comprometessem a integridade do domínio comum. As decisões seguintes não representam funcionalidades abandonadas, mas sim escolhas de âmbito para manter a primeira versão estável, testável e alinhada com o escopo principal do projeto:

- **Geração de Bridges com KSP (Kotlin Symbol Processing):** Foi ponderada a utilização de KSP para gerar automaticamente *bridges* entre Kotlin e TypeScript/Node.js. A alternativa foi descartada porque a configuração, a compatibilidade multiplataforma e os casos limite de tipagem consumiriam tempo desproporcional. Optou-se por uma fachada `@JsExport` controlada, suficiente para preparar o target JavaScript sem comprometer o prazo.
- **Injeção Automática de Ferramentas via SSE:** Também foi avaliada a possibilidade de a Gateway interceptar *tool calls* durante *streams*, executar ferramentas e injetar resultados automaticamente no fluxo. Esta opção exigiria gerir estado de conversação, suspender e retomar *streams*, lidar com concorrência e manter compatibilidade com fornecedores

que emitem eventos diferentes. A capacidade ficou identificada como trabalho futuro, mantendo a primeira versão focada em contratos, adapters, segurança, observabilidade e resiliência.

5.4 Evolução Arquitetural: De *Inference Service* a *Dispatcher*

A primeira iteração concentrava comunicação, tradução de contratos e roteamento num componente designado por `InferenceService`. Embora esta abordagem tenha sido eficaz para prototipagem, rapidamente se aproximou de um *God Object*: cada fornecedor, métrica, regra de segurança ou política de erro aumentava a responsabilidade do mesmo serviço.

O problema principal não era apenas o tamanho do componente, mas a direção das dependências. Se o serviço conhecesse diretamente contratos HTTP, clientes externos, regras de seleção de fornecedor e políticas transversais, qualquer alteração numa dessas áreas teria impacto no centro do sistema. Isto dificultaria testes unitários, tornaria mais cara a adição de novos adapters e reduziria a utilidade do SDK como biblioteca reutilizável.

A refatorização substituiu esse serviço por um contrato pequeno, o `DispatcherPort`. A responsabilidade do dispatcher é receber um `CommonRequest` e despachá-lo para uma estratégia de execução. A implementação padrão fornecida, criada por `routingDispatcherFactory`, faz um despacho simples: procura um outbound com o mesmo fornecedor e modelo presentes no pedido. Por se tratar de uma interface, o SDK não força esta estratégia; um consumidor pode substituir o dispatcher por outra implementação sem alterar inbounds ou contratos.

Esta mudança reduziu o acoplamento e clarificou a arquitetura. Os inbounds passaram a conhecer apenas a porta `DispatcherPort`; os outbounds passaram a concentrar tradução e comunicação externa; e as preocupações transversais foram deslocadas para uma estrutura própria, a *pipeline* de *interceptors*. O dispatcher passou, portanto, a ser o ponto de decisão da execução, enquanto a pipeline passou a ser apenas uma forma opcional de decorar essa execução com responsabilidades operacionais.

5.5 Evolução para a Pipeline de Interceptors

A *pipeline* foi implementada como um adapter de dispatcher. Isto é, os inbounds não recebem uma pipeline; recebem sempre um `DispatcherPort`. Quando a aplicação escolhe `useNativePipeline`, o SDK monta uma `GatewayPipeline` com interceptors e um dispatcher terminal, e depois expõe essa cadeia através de `PipelineBackedDispatcher`. Se a aplicação não quiser a pipeline, pode fornecer diretamente um dispatcher próprio através de `useCustomDispatcher`.

Esta decisão evita que a pipeline se torne uma dependência obrigatória da arquitetura. Um consumidor que pretenda apenas tradução de contratos e despacho direto pode usar um dispatcher simples. Um consumidor que precise de segurança, observabilidade e resiliência pode ativar a pipeline nativa. Um consumidor com necessidades próprias pode implementar outro `DispatcherPort`, por exemplo com seleção por custo, políticas externas, filas internas ou integração com outro orquestrador.

Modo de montagem	Componente visto pelos inbounds	Justificação
Dispatcher direto	Uma implementação de <code>DispatcherPort</code> .	Útil quando se pretende uma Gateway mínima, com tradução e envio para outbounds sem lógica transversal.
Pipeline nativa	<code>PipelineBackedDispatcher</code> , também exposto como <code>DispatcherPort</code> .	Permite aplicar autenticação, autorização, métricas, rate limiting, circuit breaker, fallback e logging sem alterar adapters.
Dispatcher customizado	Qualquer implementação externa de <code>DispatcherPort</code> .	Mantém o SDK aberto a estratégias futuras, incluindo roteamento por custo, latência, plano de cliente ou políticas de negócio.

Tabela 5.1: Formas de expor a execução aos inbounds sem os acoplar à pipeline

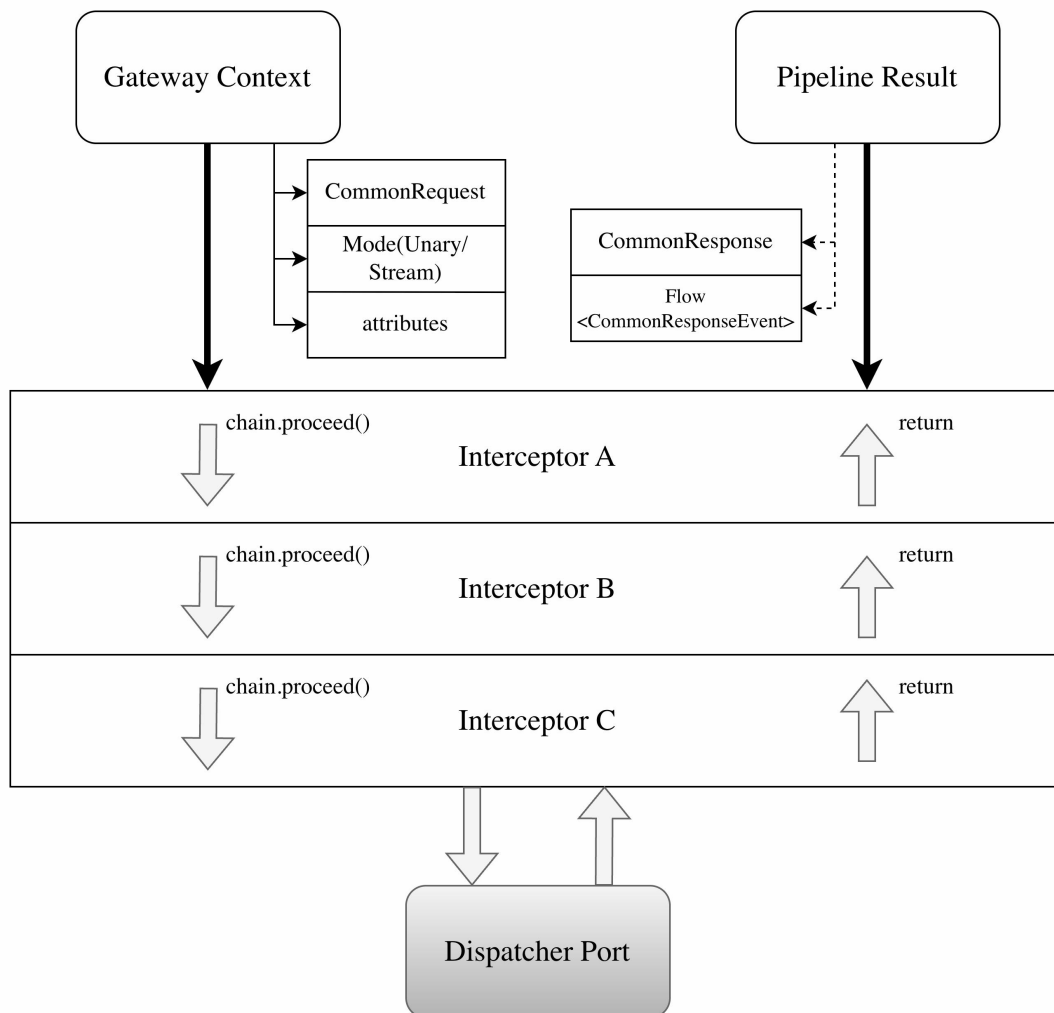


Figura 5.1: Visão conceptual da pipeline de interceptors

Esta forma de encaixe mantém a arquitetura simples. Para um caso mínimo, a Gateway pode operar com um dispatcher direto. Para um caso operacional, a pipeline permite adicio-

nar autenticação, autorização, métricas, limites, *circuit breaker*, *fallback*, logging e tracing sem alterar adapters. O modelo é semelhante ao dos filtros de Servlets [?]: cada filtro recebe um contexto, decide se continua a cadeia e pode executar lógica antes e depois do próximo elemento. A diferença é que, no OmniAI SDK, o contexto não representa apenas um pedido HTTP; representa uma chamada de inferência já normalizada, incluindo fornecedor, modelo, modo de execução, atributos tipados e eventual fluxo de *streaming*.

A *pipeline* é composta por `GatewayPipeline`, `GatewayPipelineChain`, `Interceptor` e `GatewayContext`. O `GatewayContext` transporta o pedido comum, atributos tipados e modo de execução, podendo representar chamadas síncronas ou *streaming*. A cadeia executa os *interceptors* por ordem e, no fim, invoca o dispatcher terminal.

Esta abordagem também prepara o escalamento horizontal. A pipeline não exige estado global obrigatório; cada pedido transporta o seu contexto. Onde existe estado partilhável, como rate limiting ou circuit breaker, o SDK define stores substituíveis. A implementação em memória é adequada para desenvolvimento e instância única, mas pode ser trocada por Redis, base de dados ou outro backend coordenado em implantações distribuídas. Assim, várias instâncias da OmniAI Gateway podem partilhar contadores, estados de circuito e metadados operacionais sem alterar o contrato dos inbounds nem a forma como os clientes chamam a Gateway.

5.6 Arquitetura Interna dos Interceptors

O módulo `interceptors` inclui implementações pré-construídas para preocupações operacionais e de segurança. Todos seguem o mesmo contrato: recebem um `GatewayContext`, consultam ou acrescentam atributos à `TypedMap`, decidem se chamam `chain.proceed(context)` e podem observar ou transformar o `PipelineResult`. Esta estrutura permite colocar lógica transversal antes e depois da inferência sem acoplar essa lógica aos adapters de entrada, aos adapters de saída ou ao dispatcher.

A ordem de execução é configurável. No target JVM, o SDK suporta a anotação `InterceptorPriority`, usada para executar primeiro interceptors com prioridade superior. No target JavaScript, onde a reflexão sobre anotações é limitada, a ordem segue a sequência definida pela DSL. Assim, autenticação e autorização podem ocorrer no início da cadeia, enquanto métricas, tracing e logging podem envolver a execução completa.

A motivação para estes interceptors não foi apenas organizar código, mas tornar explícitas políticas que, numa Gateway de IA, afetam segurança, custo e disponibilidade. A Tabela 5.2 resume essa relação.

Interceptor	Problema tratado	Motivo para estar na Gateway
Telemetria	Falta de visibilidade sobre latência, erros, tokens e fornecedores.	A Gateway observa todas as chamadas e consegue medir comportamento transversal sem alterar clientes ou adapters.
Autenticação	Necessidade de validar quem apresenta o token antes de consumir modelos pagos.	A Gateway é o recurso protegido e deve validar tokens antes de encaminhar tráfego para fornecedores.
Autorização	Um token válido não implica permissão para todos os modelos ou operações.	A decisão por cliente, ação, fornecedor e modelo deve ser centralizada antes da inferência.
Rate limiting	Risco de abuso, picos de carga e custos inesperados.	A Gateway é o ponto natural para aplicar quotas por cliente, plano, fornecedor ou modelo.
Circuit breaker	Dependências externas podem ficar lentas ou indisponíveis.	A Gateway evita insistir em fornecedores em falha e protege o restante sistema.
Fallback	Uma falha de fornecedor não deve, quando existe alternativa, falhar imediatamente a experiência do cliente.	A Gateway conhece os outbounds disponíveis e pode tentar alternativas mantendo o contrato do cliente.

Tabela 5.2: Justificação dos interceptors fornecidos pelo SDK

5.6.1 Interceptor de Telemetria e Observabilidade

O `MetricsInterceptor` recolhe dados operacionais essenciais num sistema que intermedeia chamadas a LLMs: latência, sucesso, erro, fornecedor, modelo, cliente e consumo de tokens. Estes dados são importantes porque uma Gateway centraliza tráfego de vários clientes e fornecedores; sem telemetria, não é possível perceber custos, degradação, falhas por modelo ou impacto de políticas como *fallback*.

A implementação distingue chamadas unárias de *streaming*. Em respostas progressivas, o interceptor não bloqueia a execução; usa operadores de fluxo para observar eventos e emitir métricas no encerramento do *stream*. Assim mede a duração total da interação, e não apenas o tempo até ao primeiro fragmento.

A arquitetura segue inversão de dependência através da porta `MetricsPort`. O OmniAI SDK define instrumentos comuns, como `counter`, `histogram` e `upDownCounter`, enquanto o adapter concreto no target JVM exporta para OpenTelemetry. O uso do OpenTelemetry Protocol (OTLP) foi escolhido porque o OTLP define a codificação, transporte e entrega de dados de telemetria entre fontes, coletores e backends [44]. Esta escolha evita prender o OmniAI SDK a Prometheus, Grafana, Datadog ou outro backend específico: o SDK emite telemetria num formato aberto, o Collector recebe e processa esses dados, e a infraestrutura decide para onde exportar.

Esta decisão é especialmente importante numa Gateway de IA. O mesmo OmniAI SDK pode ser usado por uma aplicação simples em desenvolvimento, por um POC local com Prometheus/Grafana ou por uma instalação futura com outro backend de observabilidade. A Open-

Telemetry apresenta-se precisamente como especificação aberta e agnóstica de fornecedor [40]; por isso, a instrumentação fica no OmniAI SDK, mas a escolha operacional do backend permanece fora do domínio.

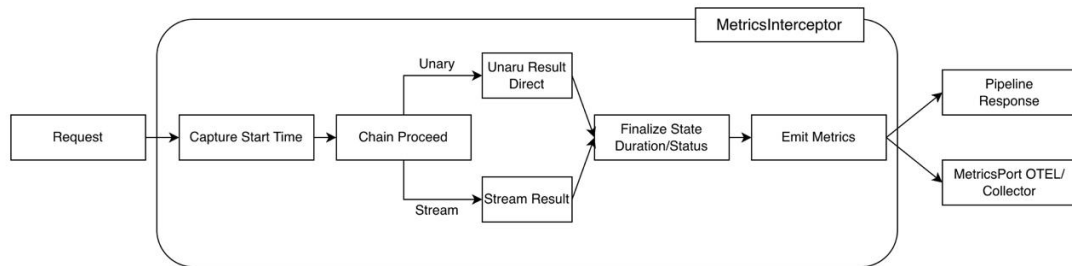


Figura 5.2: Fluxo interno de processamento e lógica de observabilidade para pedidos unários e streaming

Para adicionar uma métrica, a aplicação usa a DSL do `gateway-client`. O programador pode ativar a latência padrão, acrescentar atributos globais e definir métricas próprias com uma função `value`, que extrai o valor a partir do contexto e do resultado, e uma função `tags`, que acrescenta dimensões de análise.

```

metrics {
  metricsPort = telemetryRuntime.metricsPort
  tracer = telemetryRuntime.tracer

  attributes {
    include(ClientIpMetadataKey, alias = "client.ip")
    attribute("sdk.version") { _, _ -> "1.0.0" }
  }

  defaultLatency {
    enabled = true
    name = "gateway.inference.request.duration"
  }

  customMetrics {
    counter(
      name = "gateway.llm.tokens",
      description = "Total de tokens consumidos",
      unit = "{tokens}"
    ) {
      value { _, result ->
        (result as? PipelineResult.Unary)?.response?.usage?.totalTokens
      }
    }
  }
}

```

}

Este desenho separa três responsabilidades: a definição da métrica na DSL, a recolha no interceptor e a exportação no adapter de telemetria. Também facilita testes, porque `NoOpMetricsPort` pode ser usado quando a observabilidade está desligada.

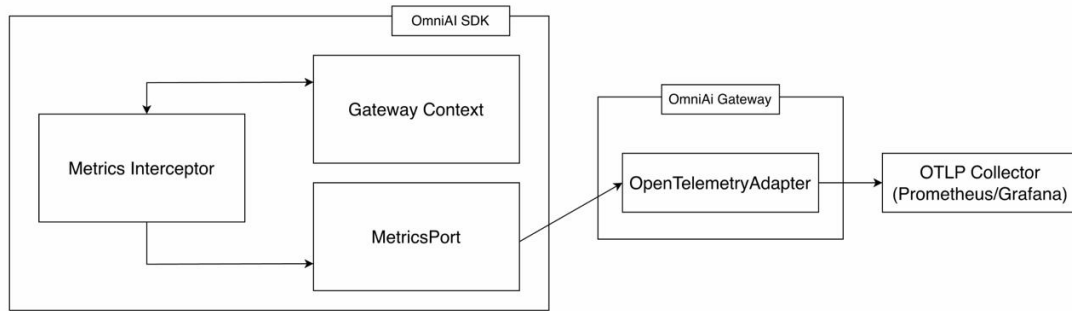


Figura 5.3: Observabilidade e métricas operacionais da OmniAI Gateway

5.6.2 Interceptor de Autenticação

A autenticação é realizada pelo **AuthenticationInterceptor**. A Gateway atua como *Resource Server*: recebe pedidos para recursos protegidos, extrai o *access token* do cabeçalho **Authorization: Bearer <token>** e valida se esse token foi emitido por um servidor de autorização confiável [45–47]. Esta responsabilidade é essencial numa AI Gateway, porque a camada central passa a proteger acesso a modelos, custos, quotas e dados operacionais.

A Gateway não deve atuar como *Authorization Server*: não emite tokens, não gere credenciais dos clientes e não substitui Logto ou Keycloak. O seu papel é o de API protegida. Por isso, valida tokens recebidos, confirma se foram emitidos para a audiência esperada e só depois permite que o pedido avance na pipeline para os próximos interceptors. Esta separação reduz responsabilidade de segurança dentro do SDK e permite integrar diferentes fornecedores de identidade sem alterar o contrato dos clientes.

O fluxo inicia-se no adapter HTTP, que propaga o token para o domínio através de metadados. O interceptor classifica o token: se tiver três segmentos, é tratado como JWT; caso contrário, como token opaco. Esta distinção permite escolher entre validação local por assinatura e validação remota por introspeção.

No modo *OIDC Discovery* (RFC 8414), a Gateway obtém dinamicamente metadados do *Authorization Server*, como *issuer*, *jwks_uri* e endpoint de introspeção [48]. Todo este fluxo encontra-se ilustrado na Figura 5.4. Para JWTs, valida o *kid*, obtém a chave pública no JWKS, verifica assinatura e confirma *claims* como *exp*, *nbf*, *iss* e *aud* [49–51]. Para tokens opacos, usa introspeção OAuth2 [52].

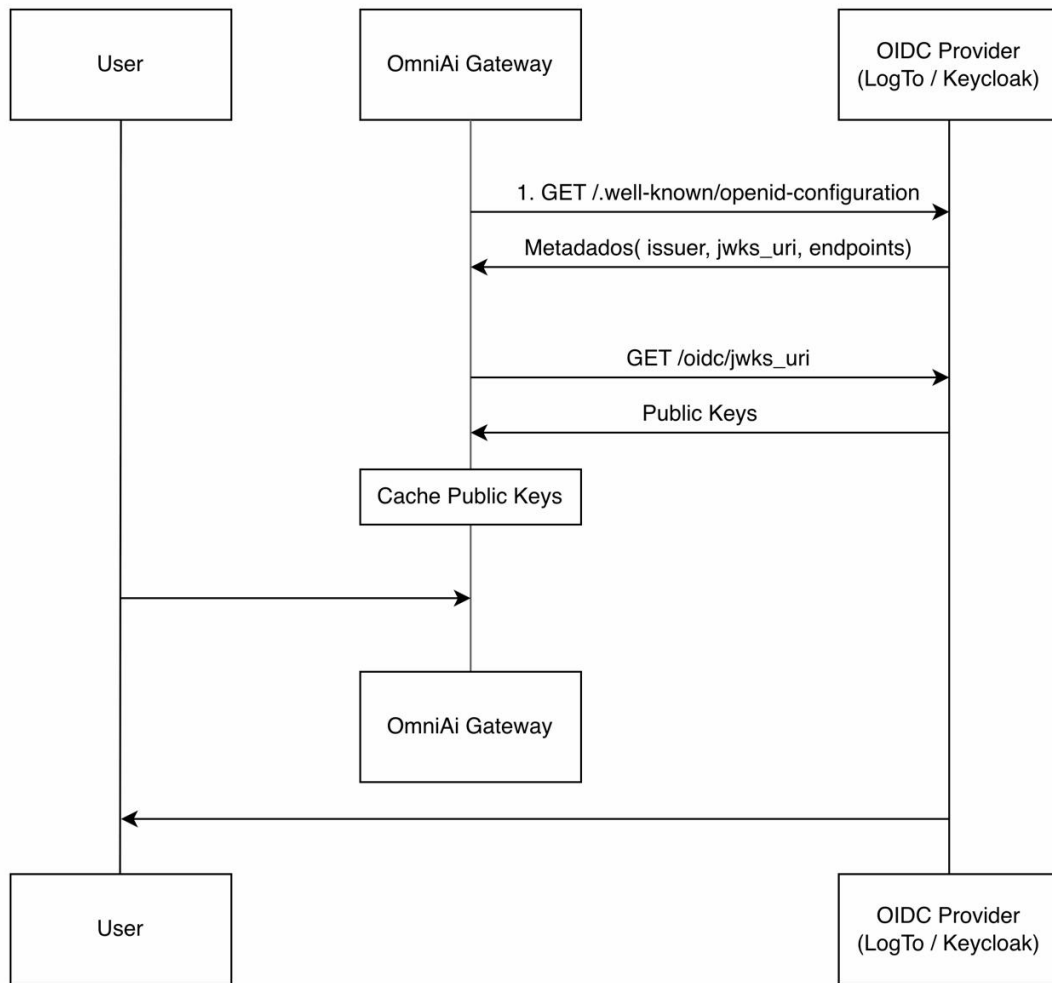


Figura 5.4: Sequência OIDC: descoberta, JWKS e cache de chaves

Para reduzir latência e chamadas repetidas ao servidor de autorização, a implementação inclui `InMemoryKeyCache`, rotação de chaves em *background* e *cooldown* para evitar consultas excessivas quando uma chave não é encontrada. No caso de tokens opacos, o `IntrospectionCache` usa *positive caching* para tokens ativos e *negative caching* para tokens inválidos. Os tokens opacos são guardados em cache através de *hash*, evitando manter credenciais em texto limpo em memória.

O resultado é encapsulado em `AuthValidationResult` e guardado na `TypedMap` sob a chave `AUTH_RESULT_KEY`. Isto permite que autorização, métricas e logging reutilizem a identidade já validada.

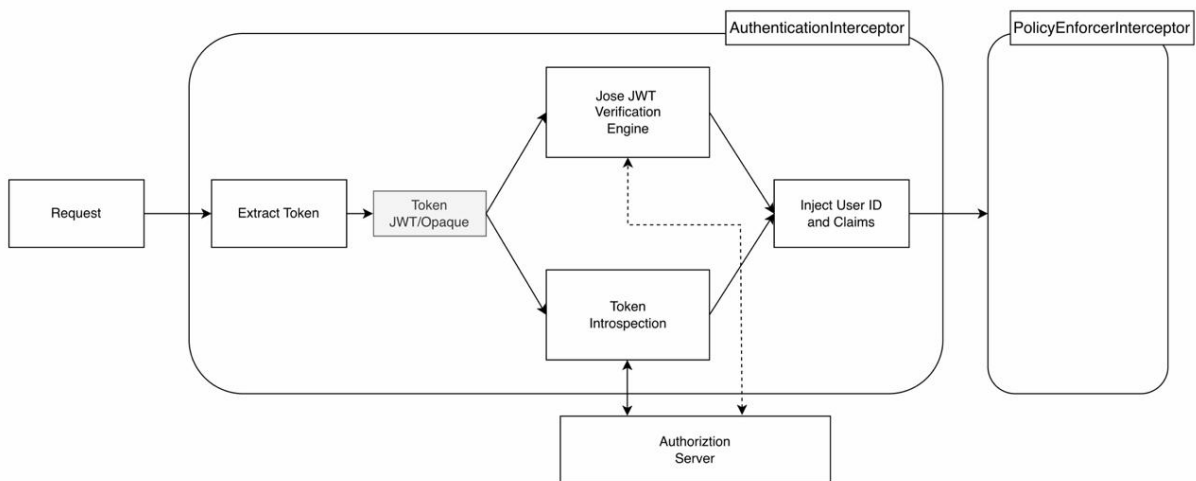


Figura 5.5: Fluxo interno do interceptor de autenticação

A Gateway final usa Logto como Authorization Server local [38], executado em Docker juntamente com PostgreSQL, OpenTelemetry Collector, Prometheus e Grafana. Foram também realizados testes exploratórios com Keycloak [53]. A escolha final do Logto no POC deveu-se à simplicidade de configuração local, suporte a aplicações Machine-to-Machine e facilidade de gerir API Resources durante a validação.

5.6.3 Interceptor de Autorização

A autorização é separada da autenticação porque responder a “quem é o utilizador?” não é o mesmo que responder a “o que este utilizador pode fazer?”. A autenticação valida identidade; a autorização decide permissões sobre uma ação e um recurso. Esta separação é importante numa Gateway multi-cliente, onde um token válido pode não ter permissão para usar todos os modelos, fornecedores ou operações. A distinção entre autenticação e autorização é também a base dos modelos modernos de identidade: a primeira prova identidade, enquanto a segunda concede permissão sobre dados ou ações protegidas [54].

No contexto do projeto, esta separação evita dois problemas. Primeiro, impede que a validação de token seja confundida com autorização total: um cliente autenticado pode estar limitado a um subconjunto de modelos, fornecedores ou quotas. Segundo, permite trocar a origem das decisões de autorização sem alterar a autenticação. Embora numa fase inicial a decisão possa operar de forma local e simples, o sistema está desenhado para que, numa iteração futura, essa mesma decisão possa ser assegurada por um PDP externo, tal como o AuthZen ou o OPA.

No OmniAI SDK, a autorização é centralizada no `PolicyEnforcerInterceptor`. O componente atua como *Policy Enforcement Point* (PEP): interceta o pedido, constrói um `AuthorizationInput` e delega a decisão a um *Policy Decision Point* (PDP) através da porta `PolicyDecisionPointPort`. Podemos ver todo este fluxo na figura 5.6.

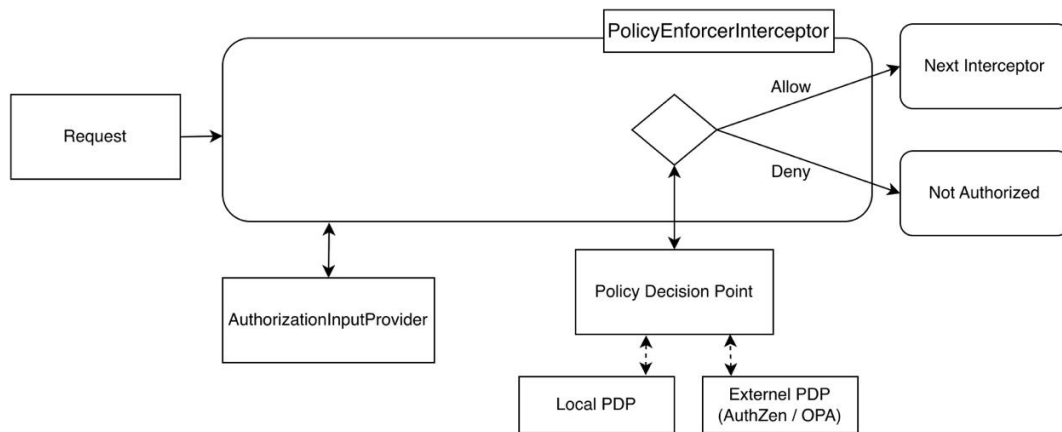


Figura 5.6: Arquitetura do interceptor de autorização: separação entre PEP e PDP

O **AuthorizationInput** segue o padrão **Subject-Action-Resource (SAR)**. O **Subject** representa o utilizador ou aplicação, incluindo identificador, roles e atributos vindos do token. A **Action** representa a operação, como `chat_completion`. O **Resource** identifica o alvo, como fornecedor ou modelo. O **Context** acrescenta atributos ambientais, como IP, tempo ou metadados do pedido. A porta do PDP permite integrar motores diferentes, incluindo políticas internas, AuthZen, OPA ou outro serviço externo. Se a decisão for *Deny*, a pipeline é interrompida antes de qualquer chamada ao fornecedor [55].

5.6.4 Interceptor de Rate Limiting

O **RateLimitInterceptor** aplica limites de utilização antes da chamada ao fornecedor. Numa AI Gateway, este mecanismo tem dois objetivos principais: proteger o sistema contra abuso ou tráfego excessivo e controlar custos associados a chamadas de inferência. A limitação de taxa é uma estratégia comum para impor um teto de ações repetidas num intervalo temporal, reduzindo abuso de APIs, força bruta e carga excessiva [56].

A importância deste interceptor é maior do que numa API tradicional, porque cada pedido pode representar custo financeiro direto, consumo de quotas externas e ocupação prolongada de ligações em *streaming*. Sem rate limiting, um cliente mal configurado, uma automação em ciclo ou um utilizador abusivo pode saturar a Gateway, esgotar quotas nos fornecedores e degradar o serviço para clientes legítimos. O interceptor permite aplicar justiça operacional: cada cliente, plano, modelo ou fornecedor pode ter limites próprios.

A lógica é orientada por políticas. Cada **RateLimitPolicy** extrai dados do **GatewayContext** e devolve **RateLimitTarget**. Um target contém uma chave, como cliente/modelo/plano, e uma **RateLimitDefinition**, composta por limite, tipo e janela temporal. Isto permite políticas diferentes por cliente, plano, modelo ou fornecedor.

O store atual em memória usa buckets protegidos por **Mutex**. Quando um pedido chega, o interceptor tenta consumir uma unidade em todos os buckets aplicáveis. Se todos permitirem consumo, a cadeia continua. Se algum limite estiver esgotado, devolve **TooManyRequestsError** com **retryAfter**. Esta resposta permite que clientes bem comportados reduzam ritmo em vez de insistirem. Para desenvolvimento e instância única, este modelo é suficiente; em escalamento

horizontal, o contrato `RateLimitStore` permite substituir a implementação por Redis ou outro backend compartilhado. Essa substituição é necessária quando várias instâncias da Gateway devem partilhar a mesma visão de quota.

5.6.5 Interceptor de Circuit Breaker

O `CircuitBreakerInterceptor` protege a Gateway contra chamadas repetidas a um outbound em falha. O padrão de *circuit breaker* é usado para lidar com falhas em serviços remotos, evitando insistir numa dependência que está indisponível e dando tempo para recuperação [57–59]. Numa Gateway de IA, esta proteção é especialmente útil porque fornecedores externos podem falhar por indisponibilidade, timeouts, quotas ou erros 5xx.

A alternativa simples seria repetir pedidos até obter sucesso. Essa abordagem é perigosa quando o fornecedor está realmente indisponível: as tentativas acumulam latência, ocupam threads ou corrotinas, aumentam custos e podem agravar a falha no serviço externo. O circuit breaker foi criado para evitar esse comportamento. Quando a falha ultrapassa um limiar, o circuito abre e a Gateway passa a falhar rapidamente aquele destino, libertando recursos e permitindo que mecanismos como fallback escolham outro outbound.

Para cada combinação de fornecedor/modelo, o interceptor identifica o `outbound.key` e consulta o `CircuitBreakerStore`. Se o circuito estiver `OPEN`, o pedido falha rapidamente com `ApiDownError` e o outbound é acrescentado ao conjunto de outbounds negados no contexto. Esta falha rápida reduz latência, evita consumo de recursos e dá ao *fallback* informação para não tentar novamente o mesmo destino.

Quando a chamada prossegue, o interceptor observa o resultado. Erros incrementam o contador de falhas; ao atingir `failureThreshold`, o circuito passa para `OPEN`. Após um período de recuperação, o estado `HALF_OPEN` permite testar um número limitado de pedidos antes de voltar ao estado `CLOSED`, evitando inundar um fornecedor que ainda está a recuperar [57]. A configuração inclui `failureThreshold`, `resetTimeoutMs` e `halfOpenTestRequests`. A implementação atual cobre a abertura e fecho básico do circuito; a evolução temporal mais completa do estado `HALF_OPEN` é uma melhoria natural.

5.6.6 Interceptor de Fallback

O `FallbackInterceptor` permite tentar alternativas quando a chamada principal falha. Em sistemas de IA, *fallback* ao nível de modelo ou fornecedor é relevante porque falhas nas APIs de inferência fornecidas pelas LLM's são um cenário frequente em produção: rate limits, indisponibilidade, quotas, timeouts ou modelos descontinuados podem impedir a resposta do fornecedor principal [60,61]. Um gateway é o ponto certo para esta política, porque conhece os outbounds configurados e consegue trocar de destino sem exigir alterações no cliente.

A decisão de colocar fallback na Gateway, e não nos clientes, reduz duplicação e melhora controlo operacional. Se cada aplicação cliente implementasse a sua própria lógica de fallback, seria necessário repetir regras de prioridade, tratamento de erros, mapeamento de contratos e métricas em vários pontos. Ao centralizar a política, o SDK consegue manter o mesmo contrato de entrada e decidir internamente que fornecedor ou modelo alternativo deve ser tentado. Isto

é particularmente útil quando o cliente usa um contrato compatível com Anthropic ou OpenAI, mas a Gateway pode executar a chamada noutro provider configurado.

A implementação atual usa a lista de outbounds disponíveis. Primeiro tenta o outbound correspondente ao fornecedor e modelo pedidos; se falhar, acrescenta esse outbound ao conjunto de negados e tenta alternativas. Para cada tentativa, cria um novo `GatewayContext` com os mesmos atributos, mas com `provider` e `model` atualizados para o outbound alternativo.

A integração com o circuit breaker é feita através do conjunto partilhado de outbounds negados. Assim, se um circuito estiver aberto, o fallback evita esse destino e tenta outro modelo ou fornecedor. Este desenho permite construir políticas de resiliência como “tentar Gemini, depois OpenAI-compatible via Groq, depois Anthropic”, mantendo a lógica de tentativa fora dos adapters. Também permite distinguir degradação controlada de erro total: a resposta pode chegar por um modelo alternativo, enquanto métricas e tracing registam que ocorreu fallback.

A evolução futura poderá acrescentar prioridades configuráveis, condições por código de erro, orçamentos de tentativa e métricas específicas de fallback. Estas regras são importantes porque nem todo erro justifica fallback: erros de autenticação ou autorização devem falhar imediatamente, enquanto timeouts, 429 e 5xx podem justificar uma alternativa.

5.6.7 Interceptor de Logging e Tracing

O `RequestLoggingInterceptor` regista fornecedor, modelo, sucesso, erro e conclusão de streams através da abstração `GatewayLogger`. No target JVM pode ser ligado a SLF4J; no target JavaScript pode usar console ou outra integração. O objetivo é garantir visibilidade mínima mesmo quando a telemetria completa não está ativa.

O logging foi mantido separado das métricas porque responde a perguntas diferentes. Métricas mostram tendências agregadas, como latência média ou número de erros; logs ajudam a explicar um pedido concreto. Numa Gateway de IA, esta distinção também tem impacto de privacidade: o objetivo é registar metadados operacionais, e não despejar prompts, respostas completas ou tokens sensíveis nos logs.

O `TracingInterceptor` envolve o pedido num *span* chamado `gateway.request.process`. Quando combinado com OpenTelemetry, este span permite correlacionar latência, erros, chamadas externas e métricas agregadas. Em cenários de fallback ou circuit breaker, tracing é particularmente útil porque permite observar o percurso real do pedido e distinguir falhas do fornecedor, decisões da Gateway e erros de cliente. Também permite perceber se uma resposta lenta resulta do fornecedor principal, de uma tentativa alternativa, de validação de token ou de uma política interna.

5.7 MCP Broker e Ferramentas Externas

O MCP Broker foi introduzido como consequência do estudo do MCP e do *tool calling*. O objetivo é separar a gestão de ferramentas externas da lógica específica das APIs de inferência. Em vez de cada adapter conhecer diretamente as ferramentas disponíveis e a forma como são executadas, o broker fornece uma camada intermédia para representar ferramentas, recursos e *prompts* de forma uniforme.

O módulo `mcp-broker` depende da biblioteca `io.modelcontextprotocol:kotlin-sdk`. A implementação atual inclui modelos para ferramentas, recursos e *prompts*, geração de esquemas JSON, configuração de transportes `stdio`, SSE e WebSocket, e mapeamento entre o domínio do projeto e tipos do SDK MCP. A classe `McpTool` descreve uma ferramenta por nome, descrição, serializador e função `execute`; `McpServerBuilder` permite registrar ferramentas, recursos e *prompts* de forma declarativa.

A Figura 5.7 representa o ciclo geral de *tool calling*. Primeiro, o cliente envia um pedido com ferramentas disponíveis. Em seguida, o modelo pode pedir uma chamada estruturada com nome e argumentos. A aplicação executa ou encaminha a ferramenta, devolve o resultado ao modelo e recebe a resposta final. O MCP Broker foi desenhado para suportar esta separação no futuro, permitindo que ferramentas sejam registradas e disponibilizadas sem acoplar cada adapter de inferência ao conjunto concreto de ferramentas.

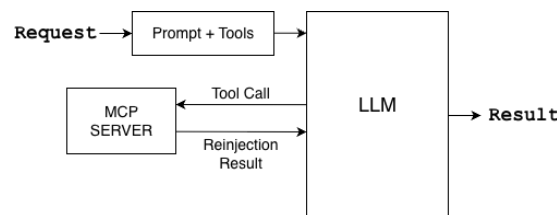


Figura 5.7: Representação do ciclo de *tool calling* e interação com ferramentas externas

No estado atual, o módulo estabelece sobretudo as abstrações, os modelos e o mapeamento necessários para essa evolução. A execução completa de ferramentas via MCP Broker, o carregamento por YAML e a integração no POC OmniAI Gateway permanecem como trabalho futuro. Esta delimitação é importante: o POC valida contratos HTTP, autenticação, telemetria e consumo do SDK, mas ainda não demonstra invocação real via MCP Broker.

5.8 Distribuição Multiplataforma

Um dos objetivos do OmniAI SDK é permitir que a lógica principal da Gateway seja reutilizada em diferentes ambientes de execução. O Kotlin Multiplatform permite separar código comum de implementações específicas: o domínio, contratos, portas, adapters principais, DSL e parte dos interceptors vivem em source sets comuns; detalhes como cliente HTTP, reflexão de prioridades, servidor Ktor e integrações de plataforma ficam em source sets específicos.

No target JVM, o OmniAI-SDK pode ser utilizado por aplicações Kotlin ou Java. Este é o target mais consolidado no estado atual do projeto e é o usado pelo POC OmniAI Gateway. A implementação JVM inclui Ktor HTTP client, integração com servidor Ktor, autenticação com descoberta OIDC, telemetria e execução real de outbounds. A distribuição futura via Maven permitirá que uma aplicação adicione o SDK como dependência versionada, sem depender de *composite build*.

No target JavaScript/Node.js, o projeto prepara uma API consumível a partir do ecossistema Node.js e TypeScript. A fachada `@JsExport` encapsula a configuração em tipos exportáveis e evita expor diretamente estruturas Kotlin que não são adequadas à fronteira JS. Esta camada é uma base para consumo via NPM, mas ainda não tem paridade total com a JVM.

A nuance principal da distribuição multiplataforma é que nem tudo deve ser comum. O domínio e os contratos devem permanecer partilhados; transporte HTTP, servidor, observabilidade e segurança podem ter implementações diferentes por plataforma. Esta divisão evita duplicação sem forçar uma abstração demasiado rígida. A linha de evolução prevista é consolidar a API pública do OmniAI-SDK, estabilizar versionamento semântico, automatizar publicação Maven/NPM por CD e documentar exemplos mínimos de consumo para JVM e Node.js.

Capítulo 6

Provas de Conceito (PoCs) e Validação

6.1 Validação do POC OmniAI Gateway

A validação principal foi realizada com a aplicação `OmniAiGateway`, um POC que consome o OmniAI SDK através do *composite build*. Esta aplicação carrega o ficheiro `application.conf`, resolve variáveis de ambiente, instancia o `KtorHttpClient`, configura outbounds, ativa segurança e telemetria, monta o runtime do SDK e expõe endpoints HTTP compatíveis com OpenAI, Anthropic e Gemini.

O comportamento validado não deve ser descrito como roteamento inteligente. A implementação atual realiza despacho configurado: o pedido contém fornecedor e modelo, e o dispatcher padrão escolhe o outbound que corresponde a essa combinação. Este desenho já permite demonstrar o desacoplamento entre cliente e fornecedor, mas políticas mais avançadas, como seleção por custo, qualidade, latência ou plano do cliente, permanecem como trabalho futuro. O trabalho de *fallback* e *circuit breaker* fornece a base técnica para esse tipo de evolução, mas o POC não afirma executar roteamento semântico ou otimizado.

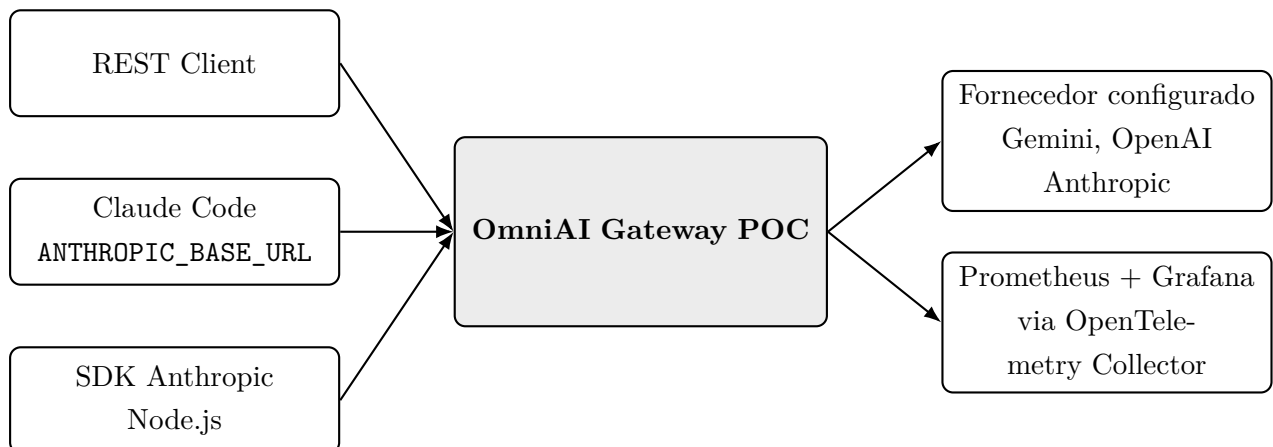


Figura 6.1: Cenários de validação do POC OmniAI Gateway

6.2 Validação com REST Client

O ficheiro `gateway-requests.http` foi usado para validar manualmente os endpoints expostos pela Gateway. Este ficheiro contém pedidos para `/v1/chat/completions`, `/v1/messages` e `/v1beta/models/{model}:generateContent`, cobrindo respostas unárias, *streaming*, geração com parâmetros e integração com token Bearer.

Este cenário foi importante por duas razões. Primeiro, permitiu validar a forma exata dos contratos HTTP recebidos pelo POC sem depender de uma aplicação externa complexa. Segundo, permitiu testar rapidamente o fluxo completo: token emitido pelo Logto, pedido autenticado, tradução inbound, despacho para outbound, chamada ao fornecedor configurado e resposta no contrato esperado pelo cliente.

6.3 Validação de Segurança com Logto

A camada de segurança foi validada com Logto como Authorization Server local. O ambiente Docker inclui Logto, PostgreSQL, OpenTelemetry Collector, Prometheus e Grafana. A configuração do POC usa OIDC Discovery para obter metadados do servidor de autorização e validar tokens com audiência configurada para o recurso da API.

Durante o desenvolvimento foram realizados testes exploratórios com Keycloak, mas a configuração final mudou para Logto por reduzir a complexidade operacional do POC. O Logto facilitou a criação de API Resources, aplicações Machine-to-Machine e fluxos OIDC adequados à validação da Gateway como Resource Server.

O fluxo também foi validado com `run_claude.py`. Este script abre o browser para autenticação no Logto, usa PKCE, troca o código por um token e inicia o Claude Code com `ANTHROPIC_BASE_URL` a apontar para a OmniAI Gateway. O cabeçalho `Authorization` é injetado através de `ANTHROPIC_CUSTOM_HEADERS`, permitindo testar um cliente real compatível com Anthropic a passar pela Gateway.

6.4 Validação de Observabilidade

A observabilidade foi validada com OpenTelemetry Collector, Prometheus e Grafana. O POC ativa métricas na DSL do SDK, incluindo latência padrão, contagem de tokens e contagem de erros. Também acrescenta atributos como IP do cliente, versão do SDK, audiência e dados derivados do resultado de autenticação, como `sub`, `email` ou `preferred_username` quando disponíveis.

O objetivo desta validação foi confirmar que a Gateway consegue produzir telemetria em tempo real enquanto processa pedidos reais. A utilização de Prometheus e Grafana permitiu observar métricas operacionais da aplicação e confirmar que a instrumentação está separada da lógica de inferência, sendo configurada através da DSL e exportada por adapters próprios.

6.5 Validação com Aplicações Cliente

Foram estudados e testados cenários com clientes que permitem alterar o endpoint da API. No caso do Claude Code, a variável `ANTHROPIC_BASE_URL` permite redirecionar pedidos para a Gateway.

Foi criado um teste com o sdk oficial da anthropic em `node-tests/anthropicSDKTest.js`, configurado com `baseUrl` local e chave fictícia, para validar que um cliente existente consegue falar com a OmniAI Gateway quando esta expõe um contrato compatível.

Também foram estudadas aplicações como OpenCode, LibreChat e Dify, por permitirem configurar endpoints compatíveis com fornecedores existentes. Estes estudos demonstram o valor arquitetural da Gateway: quando o cliente já segue um contrato conhecido, a introdução da OmniAI Gateway pode ser feita por configuração de endpoint, sem alterar a lógica principal da aplicação cliente.

6.6 Limites da Validação

A validação realizada confirma o consumo do SDK pelo POC, a exposição HTTP com Ktor, a integração com Logto, o uso de Prometheus/Grafana e a compatibilidade com clientes reais ou semi-reais. No entanto, existem limites importantes. O POC ainda não demonstra invocação de ferramentas via MCP Broker, nem políticas avançadas de roteamento por custo, latência ou qualidade. Estes pontos dependem de trabalho futuro sobre o módulo `mcp-broker`, sobre políticas de dispatcher e sobre stores distribuídos para cenários multi-instância.

Capítulo 7

Conclusões e Trabalho Futuro

7.1 Conclusões

O projeto evoluiu de uma Gateway específica para um SDK reutilizável capaz de instanciar gateways de IA configuráveis. Esta evolução tornou a solução mais alinhada com o problema identificado: reduzir acoplamento entre aplicações cliente e fornecedores de LLMs, centralizando contratos, autenticação, autorização, observabilidade, resiliência e integração com ferramentas.

O OmniAI SDK demonstra que é possível definir um domínio comum para pedidos, respostas, eventos de *streaming* e ferramentas, mantendo adapters específicos para OpenAI, Anthropic e Gemini. A Arquitetura Hexagonal ajudou a manter esta separação, enquanto a *pipeline* de *interceptors* permitiu adicionar comportamentos transversais sem contaminar o domínio.

A OmniAI Gateway, construída sobre o SDK, valida a proposta através de um POC real em Ktor, configurável por ficheiro e variáveis de ambiente, com suporte para Logto, telemetria via OpenTelemetry, Prometheus e Grafana, e integração com clientes compatíveis com Anthropic. A validação não cobre ainda invocação completa via MCP Broker nem roteamento otimizado por custo ou qualidade, mas demonstra a base arquitetural necessária para essa evolução.

7.2 Melhorias Futuras e Escalabilidade

Como trabalho futuro, existem várias linhas de evolução. A primeira é consolidar a distribuição pública do SDK via Maven e NPM, garantindo documentação de instalação, versionamento semântico e exemplos de consumo. A segunda é completar a paridade entre targets JVM e JavaScript, sobretudo na montagem de runtime e transporte HTTP.

Outra melhoria importante é aprofundar o MCP Broker, incluindo carregamento efetivo de ferramentas via YAML, execução completa dos handlers MCP e suporte robusto a SSE e WebSocket. Também será relevante melhorar as políticas de roteamento, permitindo decisões com base em custo, latência, disponibilidade, qualidade do modelo, plano do cliente e limites de utilização.

Por fim, a camada de segurança pode evoluir para políticas de autorização mais expressivas, integração com PDPs externos, introspeção real de tokens opacos e mapeamento mais fino entre claims, planos e quotas. Estas melhorias permitiriam aproximar a OmniAI Gateway de uma solução pronta para ambientes multi-tenant e produção.

Apêndice A

Exemplos JSON das APIs de Inferência

Este apêndice reúne os exemplos completos dos corpos JSON utilizados durante o estudo das APIs de inferência. Os exemplos incluem pedidos e respostas de geração de texto, chamadas de ferramentas (*tool calling*), *streaming* e configuração de raciocínio (*thinking*), permitindo ao leitor consultar as estruturas integrais que foram analisadas e comparadas no Capítulo 2.

A.1 Pedidos (*Requests*)

A.1.1 OpenAI (formato compatível)

Listing A.1: Pedido básico à API da OpenAI (geração com *system prompt*)

```
{
  "model": "llama3.2:1b",
  "messages": [
    {
      "role": "system",
      "content": "Es um assistente local a correr em Docker."
    },
    {
      "role": "user",
      "content": "Ola! Estas a funcionar?"
    }
  ],
  "temperature": 0.7
}
```

Listing A.2: Pedido à API da OpenAI com ferramentas (*tool calling*)

```
{
  "model": "llama3.2:1b",
  "messages": [
    {
      "role": "user",
      "content": "Como esta o preco das acoes da Apple (AAPL)?"
    }
  ]
}
```

```

    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_stock_price",
        "description": "Obtem o preco atual de uma acao",
        "parameters": {
          "type": "object",
          "properties": {
            "symbol": {
              "type": "string",
              "description": "O ticker da empresa, ex: AAPL"
            }
          }
        },
        "required": ["symbol"]
      }
    }
  ],
  "tool_choice": "auto"
}

```

Listing A.3: Pedido à API da OpenAI com todos os parametros de geracao

```

{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "Es um assistente tecnico que responde apenas em topicos."
    },
    {
      "role": "user",
      "content": "Explica a diferenca entre CPU e GPU."
    }
  ],
  "temperature": 0.5,
  "max_tokens": 1024,
  "top_p": 1,
  "stream": false,
  "stop": ["FIM", "TERMINAR"],
  "presence_penalty": 0.0,
  "frequency_penalty": 0.0,
  "seed": 42,
  "user": "utilizador_123",
  "response_format": { "type": "text" },
  "logit_bias": null
}

```

Listing A.4: Pedido *stream* à API da OpenAI

```
{
  "model": "llama3.2:1b",
  "messages": [
    {
      "role": "user",
      "content": "O que e Kubernetes?"
    }
  ],
  "stream": true
}
```

A.1.2 Anthropic

Listing A.5: Pedido basico à API da Anthropic (geracao com *system prompt*)

```
{
  "model": "claude-3-5-haiku-20241022",
  "max_tokens": 1024,
  "system": "You are a cat. Your name is Neko.",
  "messages": [
    {
      "role": "user",
      "content": "Hello there"
    }
  ]
}
```

Listing A.6: Pedido à API da Anthropic com ferramentas (*tool calling*)

```
{
  "model": "claude-3-5-haiku-20241022",
  "max_tokens": 1024,
  "messages": [
    {
      "role": "user",
      "content": "What is the weather like in Lisbon?"
    }
  ],
  "tools": [
    {
      "name": "get_weather",
      "description": "Get the current weather in a given location",
      "input_schema": {
        "type": "object",
        "properties": {
          "location": {
            "type": "string",
            "description": "The city and country, e.g. Lisbon, Portugal"
          }
        }
      },
      "required": ["location"]
    }
  ]
}
```



```

    }
  ]
}

```

Listing A.7: Pedido à API da Anthropic com parametros de amostragem

```

{
  "model": "claude-3-5-haiku-20241022",
  "max_tokens": 1024,
  "temperature": 1.0,
  "top_p": 0.8,
  "top_k": 10,
  "stop_sequences": ["Title"],
  "messages": [
    {
      "role": "user",
      "content": "Explain how AI works"
    }
  ]
}

```

Listing A.8: Pedido à API da Anthropic com raciocínio (*thinking*)

```

{
  "model": "claude-3-7-sonnet-20250219",
  "max_tokens": 4096,
  "thinking": {
    "type": "enabled",
    "budget_tokens": 2048
  },
  "messages": [
    {
      "role": "user",
      "content": "How does AI work?"
    }
  ]
}

```

Listing A.9: Pedido *stream* à API da Anthropic

```

{
  "model": "claude-3-5-haiku-20241022",
  "max_tokens": 200,
  "messages": [
    {
      "role": "user",
      "content": "Explain AI."
    }
  ],
  "stream": true
}

```

A.1.3 Google Gemini

Listing A.10: Pedido basico à API do Gemini (geracao com *system instruction*)

```
{
  "system_instruction": {
    "parts": [
      { "text": "You are a cat. Your name is Neko." }
    ]
  },
  "contents": [
    {
      "parts": [
        { "text": "Hello there" }
      ]
    }
  ]
}
```

Listing A.11: Pedido à API do Gemini com ferramentas (*tool calling*)

```
{
  "contents": [
    {
      "role": "user",
      "parts": [
        {
          "text": "Schedule a meeting with Bob for tomorrow."
        }
      ]
    }
  ],
  "tools": [
    {
      "functionDeclarations": [
        {
          "name": "schedule_meeting",
          "description": "Schedules a meeting with attendees.",
          "parameters": {
            "type": "object",
            "properties": {
              "attendees": {
                "type": "array",
                "items": { "type": "string" }
              },
              "date": { "type": "string" },
              "time": { "type": "string" },
              "topic": { "type": "string" }
            }
          },
          "required": ["attendees", "date", "time", "topic"]
        }
      ]
    }
  ]
}
```

```

    }
  ]
}

```

Listing A.12: Pedido à API do Gemini com configuracao de geracao e *thinking*

```

{
  "contents": [
    {
      "parts": [
        { "text": "Explain how AI works" }
      ]
    }
  ],
  "generationConfig": {
    "stopSequences": ["Title"],
    "temperature": 1.0,
    "topP": 0.8,
    "topK": 10,
    "thinkingConfig": {
      "include_thoughts": true,
      "thinkingLevel": "low"
    }
  }
}

```

Listing A.13: Pedido *stream* à API do Gemini (URI com ?alt=sse)

```

{
  "contents": [
    {
      "parts": [
        { "text": "Explain how AI works" }
      ]
    }
  ]
}

```

A.2 Respostas (*Responses*)

A.2.1 OpenAI (formato compatível)

Listing A.14: Resposta completa da API da OpenAI (geracao basica)

```

{
  "id": "chatcmpl-31",
  "object": "chat.completion",
  "created": 1773531116,
  "model": "llama3.2:1b",
  "system_fingerprint": "fp_ollama",
  "choices": [

```

```

{
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "Desculpe, mas posso tentar ajuda-lo!"
  },
  "finish_reason": "stop"
}
],
"usage": {
  "prompt_tokens": 46,
  "completion_tokens": 40,
  "total_tokens": 86
}
}

```

Listing A.15: Resposta da API da OpenAI (chamada de ferramenta)

```

{
  "id": "chatcmpl-53",
  "object": "chat.completion",
  "created": 1773531604,
  "model": "llama3.2:1b",
  "system_fingerprint": "fp_ollama",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "",
        "tool_calls": [
          {
            "id": "call_9fasug2c",
            "index": 0,
            "type": "function",
            "function": {
              "name": "get_stock_price",
              "arguments": "{\"symbol\":\"AAPL\"}"
            }
          }
        ]
      },
      "finish_reason": "tool_calls"
    }
  ],
  "usage": {
    "prompt_tokens": 171,
    "completion_tokens": 19,
    "total_tokens": 190
  }
}

```

Listing A.16: Resposta *stream* da OpenAI (eventos SSE)

```
data: {"id":"chatcmpl-308","object":"chat.completion.chunk",
      "created":1773532270,"model":"llama3.2:1b",
      "choices":[{"index":0,
        "delta":{"role":"assistant","content":"K"},
        "finish_reason":null}]}

data: {"id":"chatcmpl-308","object":"chat.completion.chunk",
      "created":1773532270,"model":"llama3.2:1b",
      "choices":[{"index":0,
        "delta":{"content":"ubernetetes "},
        "finish_reason":null}]}

data: {"id":"chatcmpl-308","object":"chat.completion.chunk",
      "created":1773532272,"model":"llama3.2:1b",
      "choices":[{"index":0,
        "delta":{},
        "finish_reason":"stop"}]}

data: [DONE]
```

A.2.2 Anthropic

Listing A.17: Resposta completa da API da Anthropic (geracao basica)

```
{
  "id": "msg_01Xf...",
  "type": "message",
  "role": "assistant",
  "model": "claude-3-5-haiku-20241022",
  "content": [
    {
      "type": "text",
      "text": "Artificial intelligence (AI) is a complex field..."
    }
  ],
  "stop_reason": "end_turn",
  "usage": {
    "input_tokens": 14,
    "output_tokens": 350
  }
}
```

Listing A.18: Resposta da API da Anthropic (chamada de ferramenta)

```
{
  "id": "msg_01Cd...",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
```

```

    "text": "I can help you check the weather in Lisbon."
  },
  {
    "type": "tool_use",
    "id": "toolu_01A02B03C",
    "name": "get_weather",
    "input": { "location": "Lisbon, Portugal" }
  }
],
"stop_reason": "tool_use"
}

```

Listing A.19: Resposta da API da Anthropic (raciocínio *thinking*)

```

{
  "id": "msg_01xyz...",
  "type": "message",
  "role": "assistant",
  "model": "claude-3-7-sonnet-20250219",
  "content": [
    {
      "type": "thinking",
      "thinking": "Here I need to explain how AI works...",
      "signature": "zxcvbnm1234567890..."
    },
    {
      "type": "text",
      "text": "Artificial Intelligence (AI) works fundamentally by..."
    }
  ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 13,
    "output_tokens": 850
  }
}

```

Listing A.20: Resposta *stream* da Anthropic (eventos SSE)

```

event: message_start
data: {"type": "message_start",
  "message": {"id": "msg_d6d...", "role": "assistant",
    "model": "claude-3-5-haiku-20241022"}}

event: content_block_start
data: {"type": "content_block_start", "index": 0,
  "content_block": {"type": "text"}}

event: content_block_delta
data: {"type": "content_block_delta", "index": 0,
  "delta": {"type": "text_delta", "text": "Artificial"}}

```

```

event: content_block_delta
data: {"type": "content_block_delta", "index": 0,
      "delta": {"type": "text_delta", "text": " intelligence"}}

event: content_block_stop
data: {"type": "content_block_stop", "index": 0}

event: message_stop
data: {"type": "message_stop"}

```

Listing A.21: Devolucao de resultado de ferramenta na API da Anthropic

```

{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01A02B03C",
      "content": "The weather is 18C and sunny."
    }
  ]
}

```

A.2.3 Google Gemini

Listing A.22: Resposta completa da API do Gemini (geracao basica)

```

{
  "candidates": [
    {
      "content": {
        "parts": [
          {
            "text": "At its simplest level, AI works by...",
            "thoughtSignature": "..."
          }
        ],
        "role": "model"
      },
      "finishReason": "STOP",
      "index": 0
    }
  ],
  "usageMetadata": {
    "promptTokenCount": 5,
    "candidatesTokenCount": 969,
    "totalTokenCount": 1480,
    "promptTokensDetails": [
      { "modality": "TEXT", "tokenCount": 5 }
    ],
    "thoughtsTokenCount": 506
  },
}

```

```

"modelVersion": "gemini-3-flash-preview",
"responseId": "feKyafqAEJ_VvdIP5s7TSA"
}

```

Listing A.23: Resposta da API do Gemini (chamada de ferramenta)

```

{
  "candidates": [
    {
      "content": {
        "parts": [
          {
            "functionCall": {
              "name": "schedule_meeting",
              "args": {
                "time": "10:00",
                "date": "2025-03-27",
                "attendees": ["Bob", "Alice"],
                "topic": "Q3 planning"
              },
              "id": "fcaog9xf"
            },
            "thoughtSignature": "..."
          }
        ],
        "role": "model"
      },
      "finishReason": "STOP",
      "index": 0,
      "finishMessage": "Model generated function call(s)."
    }
  ],
  "usageMetadata": {
    "promptTokenCount": 203,
    "candidatesTokenCount": 54,
    "totalTokenCount": 423,
    "promptTokensDetails": [
      { "modality": "TEXT", "tokenCount": 203 }
    ],
    "thoughtsTokenCount": 166
  },
  "modelVersion": "gemini-3-flash-preview",
  "responseId": "AueyadTFGvKlvdIPrv74QA"
}

```

Listing A.24: Resposta *stream* do Gemini (padrao SSE)

```

data: {"candidates": [
  {"content": {"parts": [
    {"text": "AI works by "}
  ], "role": "model"}, "index": 0}
], "usageMetadata": {
  "promptTokenCount": 4,

```



```

    "candidatesTokenCount": 25,
    "totalTokenCount": 608,
    "thoughtsTokenCount": 579
  }, "modelVersion": "gemini-3-flash-preview"}

data: {"candidates": [
  {"content": {"parts": [
    {"text": "processing data."}
  ], "role": "model"}, "index": 0}
], "usageMetadata": {
  "promptTokenCount": 4,
  "candidatesTokenCount": 51,
  "totalTokenCount": 634,
  "thoughtsTokenCount": 579
}, "modelVersion": "gemini-3-flash-preview"}

data: {"candidates": [
  {"content": {"parts": [
    {"text": "", "thoughtSignature": "..."}
  ], "role": "model"},
  "finishReason": "STOP", "index": 0}
], "usageMetadata": {
  "promptTokenCount": 4,
  "candidatesTokenCount": 853,
  "totalTokenCount": 1436,
  "thoughtsTokenCount": 579
}, "modelVersion": "gemini-3-flash-preview"}

```

Apêndice B

Histórico de Planeamento e Riscos

Durante a fase de planeamento foram identificados riscos técnicos e operacionais que continuam relevantes para a evolução do projeto.

B.1 Riscos

Durante o desenvolvimento do projeto foram identificados vários riscos técnicos e operacionais. Estes riscos influenciaram decisões de desenho, prioridades de implementação e estratégias de validação.

- **Curva de aprendizagem tecnológica:** O projeto exigiu a utilização de tecnologias e conceitos com elevada complexidade, nomeadamente Kotlin Multiplatform, arquitetura hexagonal, APIs de inferência de diferentes fornecedores, autenticação baseada em OIDC/OAuth2 e o Model Context Protocol. Esta diversidade tecnológica representou um risco inicial, porque aumentou o tempo necessário para compreender os contratos externos, configurar o ambiente de desenvolvimento e definir uma arquitetura estável. A mitigação passou pela realização de protótipos incrementais, pelo estudo isolado das APIs de OpenAI, Anthropic e Gemini, e pela separação progressiva do sistema em módulos com responsabilidades claras.
- **Instabilidade de serviços externos:** A Gateway depende de serviços de terceiros, como APIs de inferência, servidores de autorização e ferramentas externas. Estes serviços podem apresentar indisponibilidade temporária, alterações nos contratos, limites de utilização ou comportamentos diferentes dos documentados. Este risco poderia comprometer a validação da solução e tornar os testes dependentes de fatores externos. Para reduzir este impacto, foram usados testes locais, servidores simulados, mocks e abstrações sobre o transporte HTTP. Além disso, a arquitetura inclui mecanismos de resiliência, como *fallback* e *circuit breaker*, que permitem lidar melhor com falhas de fornecedores.
- **Custos de utilização:** A execução de testes com modelos pagos pode gerar custos inesperados, especialmente em cenários de *streaming*, chamadas repetidas ou testes de integração. Este risco é relevante porque a Gateway incentiva a experimentação com múltiplos fornecedores e modelos. A mitigação passou por limitar o número de chamadas reais, utilizar

fornecedores com quotas gratuitas sempre que possível, recorrer a testes locais e validar grande parte da lógica através de contratos, adapters e servidores simulados.

- **Complexidade de normalização entre fornecedores:** As APIs de OpenAI, Anthropic e Gemini partilham conceitos semelhantes, como mensagens, modelos, parâmetros de geração e ferramentas, mas representam esses conceitos de formas diferentes. Alguns elementos, como *tool calling*, *streaming*, multimodalidade e metadados específicos, não têm correspondência direta entre fornecedores. Este risco poderia levar a uma abstração demasiado rígida ou à perda de informação específica de cada API. Para o mitigar, foi definido um domínio comum extensível, complementado por campos de opções específicas do fornecedor. Desta forma, o SDK consegue representar o núcleo comum sem impedir a passagem de características particulares de cada API.
- **Segurança e dados sensíveis:** A centralização dos pedidos numa Gateway aumenta a responsabilidade sobre autenticação, autorização, gestão de tokens, logs, métricas e chaves de API. Um erro nesta camada poderia expor dados sensíveis ou permitir acesso indevido a modelos e recursos. A mitigação passou pela introdução de uma pipeline de segurança com autenticação e autorização separadas, validação de tokens, integração com um Authorization Server e cuidado na recolha de métricas e logs. Ainda assim, este risco continua relevante para uma evolução futura em direção a ambientes de produção e cenários multi-tenant.
- **Âmbito e complexidade funcional:** O domínio das Gateways de IA inclui várias funcionalidades possíveis, como roteamento por custo, políticas avançadas de autorização, execução automática de ferramentas, suporte completo a MCP, cache semântica e gestão de quotas por utilizador. Implementar todas estas capacidades numa primeira versão aumentaria significativamente o risco de atrasos e instabilidade. A mitigação passou pela definição de um âmbito inicial mais controlado, focado na arquitetura base, nos adapters principais, na pipeline de interceptors e numa Gateway funcional construída sobre o SDK. As funcionalidades mais avançadas foram identificadas como trabalho futuro.

B.2 Planeamento

Semana	Período	Descrição
1	18 fev – 22 fev (parcial)	Estudo das APIs de inferência e protocolo MCP.
2	23 fev – 1 mar	Continuação da semana anterior e início da proposta.
3	2 mar – 8 mar	Entrega Proposta do projeto
4	9 mar – 15 mar	Configuração inicial da biblioteca, definição da arquitetura base e das interfaces.
5	16 mar – 22 mar	Integração com as APIs de inferência. Foco inicial em dois fornecedores estáveis.
6	23 mar – 29 mar	Integração do MCP no OmniAI Core. Invocação e registo dinâmico de ferramentas, gerindo o ciclo de <i>tool calling</i> .
7	30 mar – 5 abr	Implementação de despacho configurável de modelos e mecanismos de redundância (<i>fallback</i> e <i>retry</i>).
8	6 abr – 12 abr	Implementação dos mecanismos de segurança, como controlo de acesso, preparação do contexto e remoção de dados sensíveis.
9	13 abr – 19 abr	Conclusão das implementações anteriores.
10	20 abr – 26 abr	Apresentação de Progresso
11	27 abr – 3 mai	Início do desenvolvimento da aplicação final OmniAI Gateway. Exposição da WEB API.
12	4 mai – 10 mai	Continuação da semana anterior e preparação das abstrações MCP para integração futura.
13	11 mai – 17 mai	Término de todas as APIs de comunicação para o exterior.
14	18 mai – 24 mai	Testes de integração do Core e da Gateway.
15	25 mai – 31 mai	Entrega da Versão beta
16	1 jun – 7 jun	Refinamento de testes a todos os módulos.
17	8 jun – 14 jun	Refinamento de código, desempenho e limpeza de bugs do sistema.
18	15 jun – 21 jun	Continuação da semana anterior e refinamento do relatório final.
19	22 jun – 28 jun 2026	Conclusão do relatório final.
20	29 jun – 5 jul 2026	Vistoria completa a todo o sistema e polimento de falhas para a entrega da Versão final.
21	6 jul – 11 jul 2026 (parcial)	Entrega Final do Projeto e Relatório

Tabela B.1: Planeamento semanal do projeto

Referências

- [1] OpenAI, “Openai api documentation.” [Online]. Available: <https://platform.openai.com/docs/>
- [2] Anthropic, “Anthropic messages api documentation.” [Online]. Available: <https://docs.anthropic.com/en/api/messages>
- [3] Google, “Gemini api documentation.” [Online]. Available: <https://ai.google.dev/docs>
- [4] IBM, “What is tool calling?” [Online]. Available: <https://www.ibm.com/think/topics/tool-calling>
- [5] DAIR.AI, “Function calling and tool use in LLM agents.” [Online]. Available: <https://www.promptingguide.ai/agents/function-calling>
- [6] KMP, “Kmp documentation.” [Online]. Available: <https://kotlinlang.org/multiplatform/>
- [7] A. D. Blog, “Hexagonal architecture: Understanding ports and adapters.” [Online]. Available: <https://medium.com/@akdevblog/hexagonal-architecture-understanding-ports-4020009e1aad>
- [8] GeeksforGeeks, “Java servlet filter.” [Online]. Available: <https://www.geeksforgeeks.org/java/java-servlet-filter/>
- [9] —, “Servlet - filterchain.” [Online]. Available: <https://www.geeksforgeeks.org/java/servlet-filterchain/>
- [10] Model Context Protocol, “Model context protocol specification.” [Online]. Available: <https://modelcontextprotocol.io>
- [11] Claude, “Get started with claude code.” [Online]. Available: <https://claude.com/product/claude-code>
- [12] IBM, “What are large language models (LLMs)?” [Online]. Available: <https://www.ibm.com/think/topics/large-language-models>
- [13] NVIDIA, “Ai tokens explained.” [Online]. Available: <https://blogs.nvidia.com/blog/ai-tokens-explained/>
- [14] Groq, “Groq api documentation.” [Online]. Available: <https://console.groq.com/docs>

- [15] IBM, “What are agentic workflows?” [Online]. Available: <https://www.ibm.com/think/topics/agentic-workflows>
- [16] CloudZero, “Ai vendor lock-in: How ai is creating a new dependency problem.” [Online]. Available: <https://www.cloudzero.com/blog/ai-vendor-lock-in/>
- [17] S. P, “System design of api gateway.” [Online]. Available: <https://medium.com/@lazygeek78/system-design-of-api-gateway-e8924b71b7fb>
- [18] Vasanthan, “Ai gateway pattern: Centralized model access layer.” [Online]. Available: <https://medium.com/@vasanthancomrads/ai-gateway-pattern-centralized-model-access-layer-c5049e4f151f>
- [19] K. Stoney, “Building an ai gateway.” [Online]. Available: <https://karlstoney.com/building-an-ai-gateway/>
- [20] Berri AI, “LiteLLM: Call all llm apis using the openai format.” [Online]. Available: <https://www.litellm.ai/>
- [21] OpenRouter, “OpenRouter: A unified interface for llms.” [Online]. Available: <https://openrouter.ai/>
- [22] JetBrains, “Kotlin multiplatform documentation.” [Online]. Available: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/get-started.html>
- [23] OpenAI, “Openai api documentation: Structured outputs and function calling (json schema).” [Online]. Available: <https://platform.openai.com/docs/guides/structured-outputs>
- [24] Anthropic, “Anthropic docs: Define tools.” [Online]. Available: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools>
- [25] JSON Schema, “Json schema specification.” [Online]. Available: <https://json-schema.org/specification>
- [26] OpenAPI Initiative, “Openapi specification v3.0.3.” [Online]. Available: <https://spec.openapis.org/oas/v3.0.3>
- [27] Google, “Gemini api reference: Schema object (openapi 3.0 subset).” [Online]. Available: <https://ai.google.dev/api/caching?#Schema>
- [28] Ollama, “Ollama documentation.” [Online]. Available: <https://ollama.com/>
- [29] Google, “Google ai studio documentation.” [Online]. Available: <https://aistudio.google.com/app/docs>
- [30] Model Context Protocol, “Architecture overview.” [Online]. Available: <https://modelcontextprotocol.io/docs/learn/architecture>
- [31] —, “Kotlin sdk for model context protocol.” [Online]. Available: <https://github.com/modelcontextprotocol/kotlin-sdk>

- [32] GeeksforGeeks, “Hexagonal architecture - system design.” [Online]. Available: <https://www.geeksforgeeks.org/system-design/hexagonal-architecture-system-design/>
- [33] Netflix Technology Blog, “Ready for changes with hexagonal architecture,” 2020. [Online]. Available: <https://netflixtechblog.com/ready-for-changes-with-hexagonal-architecture-b315ec967749>
- [34] H. Graça, “Ddd, hexagonal, onion, clean, cqrs, ... how i put it all together,” 2017. [Online]. Available: <https://herbertograca.com/2017/11/16/explicit-architecture-01-ddd-hexagonal-onion-clean-cqrs-how-i-put-it-all-together/>
- [35] Gradle, “Composite builds.” [Online]. Available: https://docs.gradle.org/current/userguide/composite_builds.html
- [36] Kotlin Serialization , “Kotlin serialization documentation.” [Online]. Available: <https://kotlinlang.org/docs/serialization.html>
- [37] Ktor, “Ktor serialization documentation.” [Online]. Available: <https://ktor.io/docs/welcome.html>
- [38] Logto, “Logto documentation.” [Online]. Available: <https://docs.logto.io/>
- [39] Postgresql, “Postgresql documentation.” [Online]. Available: <https://www.postgresql.org/docs/>
- [40] OpenTelemetry, “What is opentelemetry?” [Online]. Available: <https://opentelemetry.io/docs/what-is-opentelemetry/>
- [41] Prometheus, “Prometheus documentation.” [Online]. Available: https://prometheus.io/docs/prometheus/latest/getting_started/
- [42] Grafana, “Grafana documentation.” [Online]. Available: <https://grafana.com/docs/>
- [43] GitLab, “Pipelines ci documentation.” [Online]. Available: <https://docs.gitlab.com/ci/pipelines/>
- [44] OpenTelemetry, “Opentelemetry protocol specification.” [Online]. Available: <https://opentelemetry.io/docs/specs/otlp/>
- [45] D. Hardt, “RFC 6749: The oauth 2.0 authorization framework.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6749>
- [46] M. Jones and D. Hardt, “RFC 6750: The oauth 2.0 authorization framework: Bearer token usage.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6750>
- [47] OAuth.com, “The resource server.” [Online]. Available: <https://www.oauth.com/oauth2-servers/the-resource-server/>
- [48] M. Jones, N. Sakimura, and J. Bradley, “RFC 8414: Oauth 2.0 authorization server metadata.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8414>

- [49] M. Jones, J. Bradley, and N. Sakimura, “RFC 7519: Json web token (JWT).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [50] M. Jones, “RFC 7517: Json web key (JWK).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7517>
- [51] M. Jones, J. Bradley, and N. Sakimura, “RFC 7515: Json web signature (JWS).” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7515>
- [52] J. Richer, “RFC 7662: OAuth 2.0 token introspection.” [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7662>
- [53] Keycloak, “Keycloak documentation.” [Online]. Available: <https://www.keycloak.org/documentation>
- [54] Microsoft, “Authentication vs. authorization.” [Online]. Available: <https://learn.microsoft.com/en-us/entra/identity-platform/authentication-vs-authorization>
- [55] Identity Beyond Borders, “The four horsemen of authorization: The p’s, pep, pdp, pap, and pip.” [Online]. Available: <https://medium.com/identity-beyond-borders/the-four-horsemen-of-authorization-the-p-ps-pep-pdp-pap-and-pip-42717e445ce7>
- [56] Cloudflare, “O que é limitacao de taxa?” [Online]. Available: <https://www.cloudflare.com/pt-br/learning/bots/what-is-rate-limiting/>
- [57] Microsoft, “Circuit breaker pattern.” [Online]. Available: <https://learn.microsoft.com/azure/architecture/patterns/circuit-breaker>
- [58] Itau Tecnologia, “Compreendendo o conceito de circuit breaker e sua aplicacao em arquitetura de microsservicos.” [Online]. Available: <https://medium.com/@itautecnologia/compreendendo-o-conceito-de-circuit-breaker-e-sua-aplica%C3%A7%C3%A3o-em-arquitetura-de-microsservi%C3%A7os-108dc2d1be2c>
- [59] TreinaWeb, “O que é circuit breaker?” [Online]. Available: <https://www.treinaweb.com.br/blog/o-que-e-circuit-breaker>
- [60] T. Tombas, “Building resilient ai systems: Understanding model-level fallback mechanisms.” [Online]. Available: <https://medium.com/@tombastaner/building-resilient-ai-systems-understanding-model-level-fallback-mechanisms-436cf636045f>
- [61] Portkey, “Fallbacks.” [Online]. Available: <https://portkey.ai/docs/product/ai-gateway/fallbacks>